



**TRƯỜNG ĐẠI HỌC QUẢN LÝ VÀ CÔNG NGHỆ
THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ**

KHÓA LUẬN TỐT NGHIỆP

**Ứng dụng AI với Mô hình Ngôn ngữ lớn trong
hoạt động quản lý và vận hành doanh nghiệp**

Họ và tên sinh viên : VÕ NGỌC TRÂM ANH

Mã số sinh viên : 2201700158

Ngành đào tạo : CÔNG NGHỆ THÔNG TIN

Khóa : 2022-2026

Giảng viên hướng dẫn : TS. NGUYỄN TẤN TRUNG

Thành phố Hồ Chí Minh, tháng 04 năm 2026

LỜI CAM ĐOAN

Tác giả xin cam đoan Khóa luận tốt nghiệp do tự bản thân thực hiện và không sao chép các công trình nghiên cứu của người khác để làm sản phẩm của riêng mình. Các thông tin thứ cấp sử dụng trong Khóa luận tốt nghiệp là có nguồn gốc và được trích dẫn rõ ràng. Tác giả hoàn toàn chịu trách nhiệm về tính xác thực và nguyên bản của Khóa luận tốt nghiệp.

Tác giả

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến TS. Nguyễn Tấn Trung, giảng viên hướng dẫn khóa luận, người đã tận tình định hướng, góp ý chuyên môn và đồng hành cùng em trong suốt quá trình thực hiện đề tài. Những nhận xét sâu sắc, sự hỗ trợ kịp thời cùng tinh thần làm việc nghiêm túc của thầy đã giúp em từng bước hoàn thiện đề tài không chỉ ở khía cạnh kỹ thuật mà còn trong tư duy nghiên cứu khoa học và cách tiếp cận vấn đề một cách hệ thống.

Em xin chân thành cảm ơn Quý Thầy/Cô Khoa Công nghệ, Trường Đại học Quản lý và Công nghệ Thành phố Hồ Chí Minh, những người đã truyền đạt cho em nền tảng kiến thức quý báu trong suốt quá trình học tập. Những kiến thức về trí tuệ nhân tạo, khoa học dữ liệu, thuật toán và tư duy phân tích đã trở thành nền tảng quan trọng để em có thể triển khai và hoàn thành khóa luận này.

Em cũng xin gửi lời cảm ơn đến gia đình và bạn bè, những người luôn là chỗ dựa tinh thần vững chắc, luôn động viên, khích lệ và tạo điều kiện để em có thể tập trung hoàn thành tốt đề tài trong giai đoạn cuối của chương trình đại học.

Mặc dù đã nỗ lực hết sức, khóa luận khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu từ Quý Thầy/Cô để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn.

TRƯỜNG ĐẠI HỌC QUẢN LÝ VÀ
CÔNG NGHỆ THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ

PHIẾU CHẤM ĐIỂM
☐ ĐỒ ÁN TỐT NGHIỆP ☒ KHÓA LUẬN TỐT NGHIỆP

Họ tên người chấm điểm:

Giảng viên hướng dẫn: ☒

Giảng viên phản biện: ☐

Sinh viên/nhóm sinh viên (SV) thực hiện (Số SV trong nhóm: 1)

1. SV1: Võ Ngọc Trâm Anh

MSSV: 2201700158

Lớp: 2207A

Ngành: Công nghệ thông tin

Tên đề tài: Ứng dụng AI với Mô hình Ngôn ngữ lớn trong hoạt động quản lý và vận hành doanh nghiệp

Nhận xét

1. Hình thức:

.....
.....
.....

2. Nội dung:

.....
.....
.....
.....
.....

Kết quả đánh giá	
Điểm hình thức:	/ 10
Điểm nội dung:	/ 10
Điểm tổng kết:	/ 10

Ghi chú: Điểm tổng kết được làm tròn đến 0,5.

Đồng ý cho sinh viên báo cáo: Có ☐ Không ☐

TP. Hồ Chí Minh, ngày tháng năm 20.....

Người chấm điểm

(Ký và ghi rõ họ tên)

TRƯỜNG ĐẠI HỌC QUẢN LÝ VÀ
CÔNG NGHỆ THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ

PHIẾU CHẤM ĐIỂM
☐ ĐỒ ÁN TỐT NGHIỆP ☒ KHÓA LUẬN TỐT NGHIỆP

Họ tên người chấm điểm:

Giảng viên hướng dẫn: ☐

Giảng viên phản biện: ☒

Sinh viên/nhóm sinh viên (SV) thực hiện (Số SV trong nhóm: 1)

1. SV1: Võ Ngọc Trâm Anh

MSSV: 2201700158

Lớp: 2207A

Ngành: Công nghệ thông tin

Tên đề tài: Ứng dụng AI với Mô hình Ngôn ngữ lớn trong hoạt động quản lý và vận hành doanh nghiệp

Nhận xét

1. Hình thức:

.....
.....
.....

2. Nội dung:

.....
.....
.....
.....
.....

Kết quả đánh giá	
Điểm hình thức:	/ 10
Điểm nội dung:	/ 10
Điểm tổng kết:	/ 10

Ghi chú: Điểm tổng kết được làm tròn đến 0,5.

Đồng ý cho sinh viên báo cáo: Có ☐ Không ☐

TP. Hồ Chí Minh, ngày tháng năm 20.....

Người chấm điểm

(Ký và ghi rõ họ tên)

MỤC LỤC

MỤC LỤC

DANH MỤC THUẬT NGỮ VÀ CÁC TỪ VIẾT TẮT	i
DANH MỤC CÁC BẢNG.....	viii
DANH MỤC CÁC HÌNH	ix
MỞ ĐẦU	1
1. Lý do chọn đề tài.....	1
2. Tình hình nghiên cứu.....	1
3. Đối tượng, phạm vi nghiên cứu và ý nghĩa đề tài	2
3.1. Đối tượng nghiên cứu	2
3.2. Phạm vi nghiên cứu	2
3.3. Ý nghĩa về mặt khoa học	2
3.4. Ý nghĩa về mặt thực tiễn	2
4. Phương pháp nghiên cứu	2
4.1. Phương pháp nghiên cứu lý thuyết	2
4.2. Phương pháp nghiên cứu thực nghiệm.....	2
5. Kết cấu chung của đề tài	3
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT	4
1.1. Phát biểu bài toán	4
1.1.1. Không gian bài toán:.....	4
1.1.2. Hàm xếp hạng	4
1.1.3. Mục tiêu tối ưu.....	5
1.1.4. Các câu hỏi nghiên cứu (Research Questions – RQ).....	5
1.2. Cơ sở lý thuyết.....	6
1.2.1. Mô hình Ngôn ngữ lớn (Large Language Models – LLMs).....	6
1.2.2. Kiến trúc Retrieval-Augmented Generation (RAG).....	7
1.2.3. Hệ thống Truy xuất thông tin (IR)	8
1.2.4. Phương pháp Learning-to-Rank (LTR)	11
1.2.5. Hệ thống Đa tác tử (Multi-Agent System)	13
1.2.6. Các độ đo đánh giá.....	14
CHƯƠNG 2: PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG	16
2.1. Yêu cầu hệ thống.....	16

2.1.1. Yêu cầu chức năng.....	16
2.1.2. Yêu cầu phi chức năng.....	17
2.2. Kiến trúc tổng thể và luồng xử lý thuật toán	19
2.3. Cơ chế phối hợp và giao thức giao tiếp Đa tác tử	21
2.3.1. Chuẩn hóa dữ liệu đầu ra	22
2.3.2. Cơ chế luân chuyển và lắp ghép ngữ cảnh	22
2.3.3. Ràng buộc suy luận.....	22
2.4. Chiến lược quản trị dữ liệu	23
2.4.1. Tầng lưu trữ phục vụ tính toán nhanh (In-memory/File-based Storage)....	23
2.4.2. Tầng lưu trữ bền vững (Relational Database with JSONB)	23
2.5. Triển khai kiến trúc hệ thống	24
2.5.1. Kiến trúc xử lý bất đồng bộ (Asynchronous Architecture)	24
2.5.2. Chiến lược tải trước mô hình (Model Pre-warming).....	24
2.5.3. Cơ sở hạ tầng luồng dữ liệu thời gian thực (SSE)	24
CHƯƠNG 3: XÂY DỰNG MÔ HÌNH TRUY XUẤT ĐA TẦNG VÀ SUY LUẬN	
RÀNG BUỘC	26
3.1. Biểu diễn bài toán và kiến trúc pipeline dưới góc nhìn thuật toán	26
3.2. Mô hình truy xuất kết hợp (Hybrid Retrieval Model)	28
3.2.1. Chiến lược biểu diễn dữ liệu và phân đoạn hồ sơ.....	28
3.2.2. Khai thác tín hiệu từ vựng (Lexical Retrieval)	29
3.2.3. Khai thác tín hiệu ngữ nghĩa (Dense Retrieval)	29
3.2.4. Dung hợp thứ hạng bằng RRF	30
3.2.5. Ánh xạ từ phân đoạn sang hồ sơ.....	30
3.2.6. Vai trò trong pipeline tổng thể	31
3.3. Mô hình Learning-to-Rank và Re-ranking	31
3.3.1. Thiết kế không gian đặc trưng (Feature Engineering).....	32
3.3.2. Xếp hạng sơ bộ với LambdaMART (First-stage Re-ranking)	34
3.3.3. Tái xếp hạng ngữ nghĩa sâu với Cross-Encoder (Deep semantic Re-ranking)	35
.....	35
3.3.4. Tổ hợp điểm số và Phân ngưỡng quyết định (Final score fusion).....	35
3.4. Cơ chế suy luận và sinh ngôn ngữ có ràng buộc (Faithful RAG)	36
3.4.1. Thiết kế bối cảnh có cấu trúc (Structured context grounding)	37

3.4.2. Tối ưu hóa generation hyperparameter	37
3.4.3. Cơ chế Prompt ràng buộc	37
3.4.4. Đánh giá sơ bộ làm cơ sở thiết kế (Preliminary validation)	38
CHƯƠNG 4: THỰC NGHIỆM VÀ ĐÁNH GIÁ	39
4.1. Thiết lập thực nghiệm và ràng buộc tài nguyên	39
4.1.1. Môi trường và cấu hình thực nghiệm	39
4.1.2. Ràng buộc tài nguyên và phạm vi thực nghiệm	39
4.2. Bộ dữ liệu và chiến lược Ground Truth	40
4.2.1. Tập dữ liệu thực nghiệm và Ground Truth	40
4.2.2. Chiến lược phân chia dữ liệu (Train/Test Split)	40
4.2.3. Hệ thống đo đo đánh giá	41
4.3. Đánh giá hiệu năng Truy xuất và Xếp hạng định lượng	41
4.4. Đánh giá cơ chế sinh ngôn ngữ có ràng buộc (Faithful RAG)	43
4.5. Phân tích định tính qua trường hợp kiểm thử thực tế (qualitative case study)	44
4.5.1. Năng lực đối sánh chuyên sâu (Single CV Analysis)	44
4.5.2. Năng lực xử lý hàng loạt theo thời gian thực (Batch Streaming)	45
4.5.3. Năng lực quản trị vòng đời dữ liệu (Session Management)	46
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	47
5.1. Tổng kết và trả lời các câu hỏi nghiên cứu	47
5.2. Đóng góp của đề tài	48
5.2.1. Đóng góp về mặt khoa học	48
5.2.2. Đóng góp về mặt thực tiễn	48
5.3. Hạn chế và hướng phát triển tương lai	48
DANH MỤC TÀI LIỆU THAM KHẢO	50
PHỤ LỤC	54
PHỤ LỤC A: Đặc tả mô hình và lược đồ dữ liệu	55
PHỤ LỤC B: Đặc tả cấu hình môi trường và thư viện cài đặt	57
PHỤ LỤC C: Đặc tả giao diện lập trình ứng dụng (REST API)	59
PHỤ LỤC D: Ma trận truy vết yêu cầu và kiến trúc hệ thống	60

DANH MỤC THUẬT NGỮ VÀ CÁC TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa đầy đủ	Mô tả
AI	Artificial Intelligence	Trí tuệ nhân tạo
Ablation Study	Nghiên cứu bóc tách	Phương pháp đánh giá mô hình bằng cách loại bỏ hoặc thay đổi từng thành phần cấu trúc để đo lường mức độ đóng góp của thành phần đó
API	Application Programming Interface	Giao diện lập trình ứng dụng; tập hợp các quy tắc cho phép các thành phần phần mềm tương tác và trao đổi dữ liệu
Answer Relevancy	Sự liên quan của câu trả lời	Chỉ số đánh giá mức độ phù hợp của câu trả lời do hệ thống tạo ra so với truy vấn ban đầu, không xét đến tính đúng sai mà tập trung vào việc có giải quyết đúng trọng tâm yêu cầu hay không
ATS	Applicant Tracking System	Hệ thống theo dõi và sàng lọc hồ sơ ứng viên
Average Precision	Độ chính xác trung bình	Chỉ số đánh giá hiệu quả xếp hạng, tính bằng trung bình các giá trị độ chính xác tại mỗi vị trí mà tài liệu liên quan được tìm thấy
Backoff	Cơ chế lùi thời gian	Chiến lược đợi một khoảng thời gian tăng dần trước khi thực hiện lại một yêu cầu bị lỗi để tránh làm quá tải hệ thống
Batch	Lô dữ liệu	Một nhóm các mẫu dữ liệu được đưa vào mô hình để xử lý cùng lúc nhằm tối ưu hóa hiệu suất tính toán và tốc độ huấn luyện
Bi-Encoder	Bộ mã hóa kép	Kiến trúc mã hóa các văn bản thành các vector độc lập, cho phép tính toán độ tương đồng nhanh chóng trên quy mô dữ liệu lớn
BLEU	Bilingual Evaluation Understudy	Chỉ số đánh giá văn bản sinh tự động dựa trên độ trùng khớp từ vựng
BM25	Best Matching 25	Thuật toán truy xuất thông tin dựa trên từ khóa
Chunking	Phân đoạn dữ liệu	Kỹ thuật chia nhỏ văn bản dài thành các đoạn có kích thước cố định hoặc dựa trên ngữ nghĩa để tối ưu hóa quá trình xử lý của mô hình.

Coarse-to-fine ranking	Xếp hạng từ thô đến tinh	Chiến lược phân tầng trong tìm kiếm thông tin; bắt đầu bằng việc lọc nhanh một tập ứng viên lớn (thô) và sau đó sử dụng các mô hình phức tạp để tinh chỉnh thứ hạng (tinh)
Context Precision	Độ chính xác của ngữ cảnh	Chỉ số đo lường khả năng của hệ thống trong việc xếp hạng các tài liệu/đoạn văn bản thực sự liên quan lên các vị trí đầu trong danh sách kết quả truy xuất
Context Recall	Độ bao phủ của ngữ cảnh	Chỉ số đánh giá mức độ đầy đủ của các thông tin được truy xuất so với dữ liệu thực tế (Ground Truth) cần thiết để trả lời câu hỏi
Corpus	Tập ngữ liệu	Tập hợp các tài liệu văn bản thô được tổ chức và lưu trữ làm cơ sở dữ liệu cho việc huấn luyện hoặc truy xuất
Cosine Similarity	Độ tương đồng Cosine	Đại lượng đo lường mức độ tương quan về hướng giữa hai vector trong không gian đa chiều, thường dùng để đánh giá sự giống nhau giữa các biểu diễn văn bản
Cross-Encoder	Bộ mã hóa chéo	Kiến trúc mô hình xử lý đồng thời cặp câu đầu vào để tính toán điểm số tương quan trực tiếp, thường có độ chính xác cao hơn các bộ mã hóa độc lập
CV	Curriculum Vitae	Hồ sơ ứng viên / sơ yếu nghề nghiệp
DCG	Discounted Cumulative Gain	Chỉ số đánh giá danh sách xếp hạng có áp dụng cơ chế giảm trọng số theo vị trí
DOC/DOCX	Document	Định dạng tài liệu văn bản điện tử
Document	Tài liệu	Đơn vị dữ liệu văn bản cơ bản trong hệ thống
Domain	Miền tri thức	Phạm vi ngữ cảnh hoặc lĩnh vực chuyên biệt mà dữ liệu thuộc về, quyết định đặc thù của các thuật ngữ và khái niệm liên quan
Embedding	Nhúng (Vector hóa)	Quá trình ánh xạ các thực thể ngôn ngữ (từ, câu, đoạn văn) vào không gian vector liên tục sao cho các thực thể có ý nghĩa tương đồng sẽ nằm gần nhau

Faithfulness	Tính trung thực	Chỉ số đánh giá mức độ chính xác của nội dung được tạo ra so với nguồn dữ liệu gốc, đảm bảo không có thông tin sai lệch
FAISS	Facebook AI Similarity Search	Thư viện hỗ trợ tìm kiếm và phân cụm vector đa chiều tốc độ cao
Feature Vector	Vector đặc trưng	Tập hợp các trị số biểu diễn các thuộc tính quan trọng của một đối tượng dữ liệu dưới dạng định lượng
FR	Functional Requirement	Yêu cầu chức năng; xác định các hành động cụ thể mà hệ thống phải thực hiện
Framework	Khung làm việc	Một cấu trúc hỗ trợ có sẵn, bao gồm các thư viện, quy tắc và công cụ giúp lập trình viên xây dựng hệ thống một cách nhất quán, hiệu quả và dễ bảo trì
Generation	Khâu tạo nội dung	Quá trình sử dụng các mô hình ngôn ngữ lớn để tổng hợp và sản sinh văn bản dựa trên ngữ cảnh được cung cấp
Gradient Boosting	Tối ưu hóa Gradient	Thuật toán học máy kết hợp nhiều mô hình dự đoán yếu thành một mô hình mạnh thông qua việc tối ưu hóa hàm mất mát bằng kỹ thuật gradient
Ground Truth	Dữ liệu thực tế khách quan	Tập dữ liệu chuẩn được coi là chính xác tuyệt đối, dùng làm căn cứ để huấn luyện mô hình và đo lường sai số
Hallucination	Ảo giác thông tin	Hiện tượng mô hình ngôn ngữ tạo ra các phản hồi trông có vẻ hợp lý nhưng không có thật hoặc không dựa trên dữ liệu đầu vào
Hidden Layer	Lớp ẩn	Các lớp trung gian trong mạng nơ-ron nằm giữa lớp đầu vào (Input) và đầu ra (Output), nơi thực hiện các phép toán phức tạp để trích xuất những đặc trưng trừu tượng từ dữ liệu
HR	Human Resource	Chuyên viên nhân sự/tuyển dụng
Hybrid Retrieval	Truy xuất hỗn hợp	Phương pháp tìm kiếm kết hợp giữa truy xuất theo từ khóa (lexical) và truy xuất theo ngữ nghĩa (semantic) để tối ưu hóa kết quả trả về
Hyperparameter	Siêu tham số	Các tham số được thiết lập trước khi quá trình huấn luyện bắt đầu quyết định cách thức mô hình học từ dữ liệu

Indexing	Đánh chỉ mục	Quy trình tổ chức và sắp xếp dữ liệu theo cấu trúc tối ưu nhằm tăng tốc độ tìm kiếm và truy xuất thông tin
IR	Information Retrieval	Truy xuất thông tin
IT	Information Technology	Công nghệ thông tin
Jaccard Similarity	Độ tương đồng Jaccard	Chỉ số đo lường mức độ trùng lặp giữa hai tập hợp dữ liệu bằng cách so sánh tỷ lệ giữa phần giao và phần hợp của chúng
JD	Job Description	Mô tả công việc
JSON	JavaScript Object Notation	Định dạng dữ liệu trao đổi
JSONB	JSON Binary	Định dạng lưu trữ JSON nhị phân, giúp tối ưu không gian và tốc độ truy vấn
Lambda Gradients	Gradient Lambda	Các giá trị đạo hàm được sử dụng trong các thuật toán xếp hạng (Learning-to-Rank) để điều chỉnh trọng số dựa trên vị trí của tài liệu
Latent Space	Không gian ẩn	Không gian vector đa chiều nơi các dữ liệu thô được biểu diễn dưới dạng các đặc trưng toán học trừu tượng, giúp mô hình tính toán độ tương đồng ngữ nghĩa
Lexical Signal	Tín hiệu từ vựng	Thông tin dựa trên sự trùng khớp chính xác về mặt chữ hoặc ký tự giữa các văn bản
Listwise	Học theo danh sách	Phương pháp xếp hạng tối ưu hóa trực tiếp trên toàn bộ danh sách tài liệu
LLM	Large Language Model	Mô hình ngôn ngữ lớn
Logit	Logit	Giá trị đầu ra chưa chuẩn hóa của lớp cuối cùng trong mạng neural trước khi qua hàm Softmax, đại diện cho điểm số thô của các token ứng viên
LTR	Learning-to-Rank	Phương pháp học xếp hạng
Machine Learning	Học máy	Lĩnh vực nghiên cứu các thuật toán cho phép hệ thống tự cải thiện hiệu suất thực hiện nhiệm vụ thông qua việc học hỏi từ dữ liệu thực tế
MAP	Mean Average Precision	Chỉ số đánh giá chất lượng truy xuất
MAS	Multi-Agent System	Hệ thống đa tác tử
Metric	Độ đo/ Chỉ số đánh giá	Một tiêu chuẩn định lượng được sử dụng để đo lường, so sánh và theo dõi hiệu năng hoặc chất lượng của một

		mô hình, thuật toán hoặc quy trình cụ thể
Module	Khối chức năng	Thành phần phần mềm độc lập thực hiện một nhiệm vụ chuyên biệt trong một hệ thống lớn hơn
MRR	Mean Reciprocal Rank	Chỉ số đánh giá vị trí kết quả phù hợp đầu tiên
NDCG	Normalized Discounted Cumulative Gain	Chỉ số đánh giá chất lượng xếp hạng
Neural Network	Mạng neural nhân tạo	Mô hình điện toán mô phỏng cấu trúc hệ thần kinh sinh học, gồm nhiều lớp các đơn vị xử lý kết nối với nhau để học các biểu diễn dữ liệu phức tạp
NFR	Non-Functional Requirement	Yêu cầu phi chức năng; xác định các tiêu chuẩn vận hành của hệ thống như tính bảo mật, hiệu năng, khả năng mở rộng và độ tin cậy
Pairwise	Học theo cặp	Phương pháp xếp hạng dựa trên việc so sánh mức độ liên quan giữa từng cặp tài liệu
PDF	Portable Document Format	Định dạng tài liệu điện tử
Pipeline	Quy trình xử lý	Hệ thống các bước xử lý dữ liệu được thiết kế nối tiếp nhau theo một trình tự xác định
Precision	Độ chính xác	Tỷ lệ giữa số lượng kết quả thực sự liên quan trên tổng số kết quả mà mô hình dự đoán là liên quan (đánh giá tính đúng đắn)
Prompt	Câu lệnh điều hướng	Đoạn văn bản đầu vào được thiết kế để hướng dẫn LLM thực hiện một tác vụ cụ thể, như trích xuất thông tin hoặc sinh nhận xét
Query	Truy vấn	Yêu cầu hoặc thông tin đầu vào được người dùng cung cấp để hệ thống thực hiện việc tìm kiếm hoặc xử lý
RAM	Random Access Memory	Bộ nhớ truy xuất ngẫu nhiên, dùng để lưu trữ dữ liệu tạm thời trong quá trình máy tính xử lý
RAG	Retrieval-Augmented Generation	Sinh nội dung có tăng cường truy xuất
RAGAS	RAG Assessment	Khung đánh giá chuyên biệt cho hệ thống RAG, tập trung vào các khía

		cạnh như tính trung thực, sự liên quan của ngữ cảnh và câu trả lời
Recall	Độ bao phủ	Tỷ lệ giữa số lượng kết quả liên quan được mô hình tìm thấy trên tổng số kết quả liên quan thực tế có trong dữ liệu (đánh giá tính đầy đủ)
Recommendation	Đề xuất	Quy trình hệ thống tự động đưa ra danh sách các thực thể tiềm năng nhất dựa trên các tiêu chí tính toán
Re-ranking	Tái xếp hạng	Giai đoạn thứ hai trong hệ thống tìm kiếm, sử dụng các mô hình phức tạp (như Cross-Encoder) để tinh chỉnh thứ tự các kết quả tiềm năng đã được chọn lọc sơ bộ
REST	Representational State Transfer	Chuẩn kiến trúc thiết kế API cho các dịch vụ web
Retrieval	Truy xuất	Giai đoạn tìm kiếm và lọc ra các tài liệu phù hợp nhất từ cơ sở dữ liệu dựa trên một yêu cầu cụ thể
Retrieve-then-rerank	Truy xuất và Tái xếp hạng	Chiến lược hai tầng phổ biến trong hệ thống tìm kiếm thông tin: Tầng 1 lọc nhanh ứng viên tiềm năng, Tầng 2 sắp xếp lại để đạt độ chính xác tối ưu
Retry	Cơ chế thử lại	Quá trình hệ thống tự động thực hiện lại một tác vụ khi gặp lỗi tạm thời
ROUGE	Recall-Oriented Understudy for Gisting Evaluation	Chỉ số đánh giá văn bản sinh tự động tập trung vào độ bao phủ
RQ	Research Question	Câu hỏi nghiên cứu mà đề tài tập trung giải quyết và chứng minh thông qua thực nghiệm
RRF	Reciprocal Rank Fusion	Phương pháp kết hợp nhiều danh sách xếp hạng
Score	Điểm số	Giá trị định lượng thể hiện mức độ tương đồng hoặc phù hợp sau khi qua các mô hình đánh giá
Self-Attention	Tự chú ý	Cơ chế cho phép mô hình tập trung vào các phần khác nhau của cùng một chuỗi đầu vào để hiểu mối quan hệ nội tại của chúng
Semantic Signal	Tín hiệu ngữ nghĩa	Thông tin dựa trên ý nghĩa và nội dung ẩn sau văn bản, thay vì chỉ dựa trên sự trùng khớp mặt chữ

Soft-normalization	Chuẩn hóa mềm	Kỹ thuật điều chỉnh các giá trị đặc trưng về một khoảng xác định (thường là $[0, 1]$) một cách linh hoạt, giúp làm mượt dữ liệu và giảm ảnh hưởng của các giá trị nhiễu (outliers)
Skill-based Signal	Tín hiệu kỹ năng	Các đặc trưng tập trung vào năng lực chuyên môn và kỹ năng thực hành được trích xuất từ văn bản
SSE	Server-Sent Events	Cơ chế truyền dữ liệu thời gian thực từ server
Streaming	Truyền tải dòng dữ liệu	Kỹ thuật truyền dữ liệu liên tục theo thời gian thực, cho phép người dùng theo dõi kết quả ngay khi chúng đang được tạo ra
Structural Signal	Tín hiệu cấu trúc	Thông tin dựa trên bố cục, định dạng và cách sắp xếp các thành phần trong một tài liệu
Temperature	Tham số nhiệt độ	Một hyperparameter điều chỉnh độ tập trung của phân phối xác suất đầu ra trong LLM
TF-IDF	Term Frequency – Inverse Document Frequency	Phương pháp biểu diễn tần suất từ
Token	Đơn vị từ vựng	Đơn vị cơ bản nhỏ nhất (như từ, cụm từ hoặc ký tự) mà mô hình ngôn ngữ có thể hiểu và xử lý sau khi thực hiện tách từ (Tokenization)
Transformer	Kiến trúc Transformer	Kiến trúc mạng neural dựa hoàn toàn trên cơ chế Attention, là nền tảng cho hầu hết các LLM hiện đại
Vector	Vector	Một cấu trúc dữ liệu gồm dãy các con số đại diện cho vị trí của một đối tượng trong không gian toán học đa chiều
Verdict	Kết luận	Kết quả đánh giá cuối cùng từ một quy trình thẩm định hoặc từ một tác nhân AI về một tiêu chí cụ thể

DANH MỤC CÁC BẢNG

Bảng 4.1. So sánh hiệu năng của các phương pháp Truy xuất và Tái xếp hạng	41
Bảng 4.2. So sánh nhóm độ đo RAGAS	43

DANH MỤC CÁC HÌNH

Hình 1.1. Minh họa kiến trúc cơ bản của hệ thống RAG	7
Hình 1.2. So sánh kiến trúc hệ thống Truy xuất thưa (Sparse Retrieval) dựa trên từ khóa và Truy xuất ngữ nghĩa (Dense Retrieval) dựa trên vector	9
Hình 1.3. So sánh kiến trúc Bi-Encoder và Cross-Encoder trong bài toán truy xuất....	12
Hình 2.1. Kiến trúc tổng thể và luồng xử lý thuật toán của hệ thống TalentOS	19
Hình 4.1. Giao diện trực quan hóa kết quả đối sánh đa chiều và suy luận AI của hệ thống đối với một hồ sơ thực tế	44
Hình 4.2. Giao diện xử lý phân tích hàng loạt tích hợp luồng dữ liệu thời gian thực (SSE)	45
Hình 4.3. Giao diện quản lý lịch sử các phiên phân tích và lưu trữ quyết định nhân sự	46

MỞ ĐẦU

1. Lý do chọn đề tài

Trong xu hướng ứng dụng trí tuệ nhân tạo (AI) vào quản lý và vận hành doanh nghiệp, hoạt động tuyển dụng ngày càng đòi hỏi cao về tốc độ xử lý, độ chính xác và khả năng hỗ trợ quyết định. Trên thực tế, đối với mỗi vị trí, doanh nghiệp có thể tiếp nhận số lượng lớn hồ sơ ứng viên trong thời gian ngắn, khiến quá trình sàng lọc trở thành một bài toán tốn kém về thời gian và nguồn lực.

Các hệ thống hỗ trợ tuyển dụng truyền thống (ATS) hiện nay chủ yếu dựa trên cơ chế đối sánh từ khóa, do đó còn hạn chế trong việc hiểu ngữ nghĩa, chưa phản ánh đầy đủ mức độ phù hợp tương đối giữa các ứng viên, cũng như thiếu đi các phân tích có tính giải thích minh bạch. Điều này đặt ra nhu cầu cấp thiết về các hệ thống thông minh hơn, có khả năng xử lý dữ liệu văn bản phi cấu trúc và hỗ trợ ra quyết định một cách nhất quán.

Xuất phát từ bối cảnh đó, đề tài lựa chọn nghiên cứu xây dựng hệ thống TalentOS, hướng tới việc ứng dụng AI và Mô hình ngôn ngữ lớn (LLMs) nhằm tự động hóa quá trình phân tích, đối sánh và xếp hạng hồ sơ ứng viên, từ đó nâng cao hiệu quả của hoạt động tuyển dụng trong môi trường doanh nghiệp hiện đại.

2. Tình hình nghiên cứu

Bài toán đối sánh giữa hồ sơ ứng viên và mô tả công việc đã được nghiên cứu theo nhiều hướng tiếp cận khác nhau. Các phương pháp truy xuất truyền thống (tiêu biểu như BM25) có ưu thế về tốc độ và khả năng bắt khớp từ khóa chuyên ngành, nhưng lại hạn chế trong việc hiểu nội dung ngữ nghĩa. Ngược lại, các phương pháp dựa trên biểu diễn ngữ nghĩa (Dense Embedding) giúp cải thiện khả năng hiểu nội dung, song vẫn gặp khó khăn trong các miền dữ liệu kỹ thuật có nhiều từ khóa đặc thù.

Gần đây, việc kết hợp truy xuất thông tin với các mô hình ngôn ngữ lớn thông qua kiến trúc Sinh văn bản có tăng cường truy xuất (Retrieval-Augmented Generation – RAG) đang trở thành xu hướng nổi bật. Hướng tiếp cận này cho phép hệ thống không chỉ tìm kiếm mà còn hỗ trợ phân tích và sinh nội dung phục vụ ra quyết định. Tuy nhiên, vấn đề về độ tin cậy của thông tin sinh ra và hiện tượng ảo giác thông tin (hallucination) vẫn là một thách thức lớn.

Do đó, nghiên cứu này hướng tới việc xây dựng một pipeline tích hợp nhiều phương pháp – bao gồm truy xuất lai (Hybrid Retrieval), cơ chế tái xếp hạng (Re-

ranking) học xếp hạng (Learning-to-Rank) và sinh văn bản – nhằm cân bằng giữa hiệu quả xếp hạng và tính trung thực (faithfulness) của kết quả đầu ra.

3. Đối tượng, phạm vi nghiên cứu và ý nghĩa đề tài

3.1. Đối tượng nghiên cứu

Đề tài lấy hoạt động tuyển dụng làm trọng tâm, cụ thể là tập trung nghiên cứu bài toán đối sánh và xếp hạng giữa hồ sơ ứng viên và mô tả công việc, đồng thời xem xét các kỹ thuật kiểm soát tính trung thực của thông tin trong kiến trúc ứng dụng LLMs.

3.2. Phạm vi nghiên cứu

Nghiên cứu được đặt trong bối cảnh tổng quát của bài toán đối sánh hồ sơ ứng viên và mô tả công việc trong tuyển dụng. Trong phạm vi thực nghiệm, đề tài sử dụng dữ liệu thuộc lĩnh vực Công nghệ thông tin nhằm đảm bảo tính đặc thù và khả năng đánh giá định lượng rõ ràng. Về mặt kỹ thuật, đề tài tập trung thiết kế kiến trúc thuật toán lõi.

3.3. Ý nghĩa về mặt khoa học

Đề tài cung cấp minh chứng thực nghiệm về hiệu quả của việc kết hợp các tín hiệu truy xuất từ vựng và ngữ nghĩa trong bài toán tuyển dụng. Đồng thời, nghiên cứu đề xuất một hướng tiếp cận nhằm cấu trúc hóa ngữ cảnh đầu vào, góp phần giảm thiểu hiện tượng ảo giác thông tin của các mô hình ngôn ngữ lớn.

3.4. Ý nghĩa về mặt thực tiễn

Kết quả của đề tài là hệ thống TalentOS hoàn chỉnh, hỗ trợ các doanh nghiệp tự động hóa quá trình sàng lọc hồ sơ hàng loạt. Hệ thống không chỉ xếp hạng ứng viên chuẩn xác mà còn tự động trích xuất các khoảng trống kỹ năng và gợi ý câu hỏi phỏng vấn, giúp giảm chi phí thời gian và nâng cao tính minh bạch trong quá trình ra quyết định tuyển dụng.

4. Phương pháp nghiên cứu

Đề tài sử dụng kết hợp hai phương pháp chính:

4.1. Phương pháp nghiên cứu lý thuyết

Về mặt lý thuyết, nghiên cứu tiến hành khảo sát và tổng hợp các công trình liên quan về Truy xuất thông tin, Học xếp hạng, Tái xếp hạng và RAG nhằm xây dựng nền tảng lý luận cho việc lựa chọn kiến trúc hệ thống.

4.2. Phương pháp nghiên cứu thực nghiệm

Về mặt thực nghiệm, đề tài xây dựng hệ thống thử nghiệm, tiến hành đo lường và đánh giá hiệu quả thông qua các độ đo chuẩn xác định (như NDCG, MAP cho phần

xếp hạng và framework RAGAS cho phần sinh văn bản), từ đó phân tích kết quả để rút ra các luận điểm khoa học.

5. Kết cấu chung của đề tài

Nội dung chính của khóa luận được trình bày trong 5 chương:

- **Chương 1. Cơ sở lý thuyết**

Chương này trình bày các nền tảng lý luận liên quan đến đề tài, bao gồm mô hình ngôn ngữ lớn (LLM), kiến trúc Retrieval-Augmented Generation (RAG), các phương pháp truy xuất thông tin, học xếp hạng và các độ đo đánh giá hệ thống. Đây là cơ sở để định vị bài toán và làm rõ các thành phần kỹ thuật được sử dụng trong nghiên cứu.

- **Chương 2. Phân tích yêu cầu và thiết kế hệ thống**

Chương này tập trung phân tích các yêu cầu chức năng, yêu cầu phi chức năng và các ràng buộc thực tế của bài toán. Trên cơ sở đó, đề tài đề xuất kiến trúc tổng thể của hệ thống TalentOS, mô tả luồng xử lý dữ liệu, kiến trúc đa tác tử (multi-agent) và chiến lược tổ chức dữ liệu trong hệ thống.

- **Chương 3. Xây dựng mô hình truy xuất đa tầng và suy luận ràng buộc**

Chương này trình bày chi tiết phương pháp đề xuất nhằm giải quyết bài toán đối sánh CV và JD. Trọng tâm của chương là xây dựng một mô hình xếp hạng đa tầng, kết hợp giữa truy xuất thông tin, học xếp hạng và suy luận ngôn ngữ có ràng buộc. Cụ thể, chương tập trung vào việc thiết kế các đặc trưng cho bài toán Learning-to-Rank, xây dựng hàm xếp hạng dựa trên nhiều nguồn tín hiệu, cũng như phân tích vai trò của các mô hình như LambdaMART và Cross-Encoder trong việc tối ưu chất lượng xếp hạng. Bên cạnh đó, chương cũng đề xuất cơ chế kiểm soát suy luận trong tầng sinh ngôn ngữ nhằm đảm bảo tính nhất quán và hạn chế hiện tượng sinh thông tin không có cơ sở.

- **Chương 4. Thực nghiệm và đánh giá**

Chương này mô tả bộ dữ liệu, quy trình thực nghiệm, các thiết lập mô hình, hệ thống độ đo và kết quả đánh giá. Trên cơ sở đó, đề tài tiến hành phân tích sâu các kết quả đạt được, chỉ ra ưu điểm, hạn chế và các phát hiện quan trọng rút ra từ thực nghiệm.

- **Chương 5. Kết luận và hướng phát triển**

Chương cuối cùng tổng kết các đóng góp chính của đề tài, nêu lên những hạn chế còn tồn tại và đề xuất một số hướng nghiên cứu, phát triển tiếp theo nhằm hoàn thiện hơn hệ thống trong tương lai.

NỘI DUNG

CHƯƠNG 1:

CƠ SỞ LÝ THUYẾT

1.1. Phát biểu bài toán

Quản trị nhân sự là một trụ cột cốt lõi trong hoạt động quản lý và vận hành doanh nghiệp. Trong bối cảnh tuyển dụng hiện đại, bài toán đối sánh giữa hồ sơ ứng viên (Curriculum Vitae – CV) và mô tả công việc (Job Description – JD) thường được tiếp cận dưới dạng phân loại hoặc chấm điểm thủ công theo từng tiêu chí. Tuy nhiên, cách tiếp cận này chưa phản ánh đúng bản chất của quy trình tuyển dụng, nơi mục tiêu cốt lõi nên là xếp hạng ứng viên theo mức độ phù hợp thay vì đưa ra quyết định nhị phân.

Trong nghiên cứu này, bài toán được tái định nghĩa dưới góc nhìn của Truy xuất thông tin (Information Retrieval – IR) cụ thể là bài toán xếp hạng tài liệu có giám sát (supervised document ranking). Khi đó, mô tả công việc được xem như một truy vấn, và mỗi CV được xem như một tài liệu cần xếp hạng.

1.1.1. Không gian bài toán:

Gọi:

- **Truy vấn (query) q :** Là văn bản tự nhiên mô tả vị trí tuyển dụng (JD), bao gồm tiêu đề vị trí, yêu cầu kỹ năng kỹ thuật, mức độ kinh nghiệm và các yêu cầu bổ sung.
- **Tài liệu (document) d_i :** Là hồ sơ ứng viên (CV) dưới dạng văn bản tự nhiên được trích xuất từ file PDF, DOC hoặc DOCX.
- **Tập ứng viên D :** Cho một tập $D = \{d_1, d_2, \dots, d_n\}$ (cần xác định ràng buộc thực tế trong chế độ batch của hệ thống nhằm tối ưu hóa giới hạn context window của LLM và độ trễ API trong một chu kỳ xử lý).

Mục tiêu của hệ thống là xác định một thứ tự xếp hạng π trên tập D đối với truy vấn q sao cho:

$$d_i > d_j \Leftrightarrow f(q, d_i) \geq f(q, d_j)$$

trong đó, ký hiệu “ $>$ ” biểu diễn quan hệ ưu tiên trong xếp hạng (preference relation), tức là ứng viên d_i được xếp hạng cao hơn d_j đối với truy vấn q .

1.1.2. Hàm xếp hạng

Mục tiêu trung tâm của bài toán là học một hàm xếp hạng:

$$f: Q \times D \rightarrow \mathbb{R}$$

trong đó, $f(q, d_i) \in [0, 100]$ biểu diễn mức độ phù hợp tổng thể của hồ sơ ứng viên d_i đối với mô tả công việc q .

Hàm f được tổng hợp từ một feature vector đa chiều $\phi(q, d) \in \mathbb{R}^k$ – trong đó k phụ thuộc vào thiết kế đặc trưng cụ thể của hệ thống – bao gồm bốn nhóm tín hiệu chính:

- **Lexical signals:** mức độ khớp từ vựng (BM25 score)
- **Semantic signals:** độ tương đồng ngữ nghĩa (cosine similarity)
- **Skill-based signals:** mức độ phù hợp kỹ năng (Jaccard similarity, skill coverage)
- **Structural signals:** đặc trưng hồ sơ (seniority, kinh nghiệm, học vấn)

Giá trị $f(q, d)$ giúp mã hóa mức độ đáp ứng yêu cầu tuyển dụng trên nhiều khía cạnh khác nhau. Trong bối cảnh nghiệp vụ, điểm số này được ánh xạ thành các mức khuyến nghị nhằm phục vụ mục đích minh họa và có thể hiệu chỉnh đáp ứng thực tế:

- $f(q, d_i) \geq 75$: shortlist
- $55 \leq f(q, d_i) < 75$: hold
- $f(q, d_i) < 55$: reject

1.1.3. Mục tiêu tối ưu

Trong thực tế, nhà tuyển dụng thường chỉ xem xét một số lượng nhỏ ứng viên ở đầu danh sách, do đó các vị trí đầu có giá trị thực tiễn cao hơn đáng kể so với các vị trí phía sau. Vì vậy, hệ thống chọn NDCG@k làm evaluation metric chính:

$$\text{NDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}, \quad \text{DCG@k} = \sum_{i=1}^k \frac{2^{r_i} - 1}{\log_2(i + 1)}$$

trong đó r_i là mức độ liên quan của CV ở vị trí thứ i .

Việc sử dụng NDCG@k cho phép ưu tiên tối đa độ chính xác các ứng viên ở vị trí đầu danh sách, hỗ trợ đánh giá mức độ phù hợp đa cấp (shortlist, hold, reject), và phản ánh sát hành vi thực tế trong quy trình tuyển dụng.

1.1.4. Các câu hỏi nghiên cứu (Research Questions – RQ)

Từ việc tái định nghĩa bài toán dưới góc nhìn IR, nghiên cứu tập trung trả lời các câu hỏi nghiên cứu (RQ) sau:

- RQ1: Phương pháp truy xuất nào mang lại hiệu quả tối ưu trong bài toán sàng lọc hồ sơ?

- RQ2: Phương pháp Learning-to-Rank có cải thiện chất lượng xếp hạng so với việc sử dụng bộ trọng số được thiết kế thủ công không?
- RQ3: Việc tích hợp Cross-Encoder có đáng để sử dụng khi xét đến sự đánh đổi giữa độ chính xác và độ trễ (latency)?
- RQ4: Làm thế nào để hệ thống RAG đảm bảo faithfulness và hạn chế hiện tượng sinh thông tin không có cơ sở (hallucination)?

Tổng thể, việc tái mô hình hóa bài toán theo hướng xếp hạng tài liệu cho phép tích hợp hiệu quả các kỹ thuật truy xuất, học xếp hạng và mô hình ngôn ngữ lớn trong một pipeline thống nhất, tạo nền tảng cho việc xây dựng hệ thống hỗ trợ tuyển dụng có tính chính xác, minh bạch và khả năng mở rộng cao.

1.2. Cơ sở lý thuyết

1.2.1. Mô hình Ngôn ngữ lớn (Large Language Models – LLMs)

Mô hình ngôn ngữ lớn (LLMs) là các mô hình học sâu quy mô lớn được xây dựng trên kiến trúc Transformer (Vaswani và cộng sự, 2017), và được huấn luyện trước trên tập dữ liệu văn bản quy mô lớn nhằm học biểu diễn ngữ nghĩa của ngôn ngữ tự nhiên. Nhờ khả năng mô hình hóa phụ thuộc ngữ cảnh thông qua cơ chế self-attention, LLMs có thể thực hiện hiệu quả nhiều tác vụ xử lý ngôn ngữ như trích xuất thông tin, tóm tắt và sinh văn bản.

Khác với các mô hình truy xuất hoặc xếp hạng truyền thống, LLMs đóng vai trò như một module suy luận ngôn ngữ (reasoning module), có khả năng phân tích, tổng hợp và sinh nội dung dựa trên ngữ cảnh đầu vào. Điều này cho phép LLMs không chỉ xử lý văn bản ở mức bề mặt mà còn nắm bắt được các quan hệ ngữ nghĩa phức tạp và suy diễn thông tin.

Trong phạm vi đề tài, LLMs được sử dụng như một thành phần suy luận trong pipeline hệ thống. Cụ thể, LLMs đảm nhận hai vai trò chính: (i) trích xuất và chuẩn hóa thông tin từ CV và JD sang dạng dữ liệu có cấu trúc, và (ii) thực hiện suy luận ngữ nghĩa nhằm phân tích mức độ phù hợp, xác định khoảng trống kỹ năng và sinh các câu hỏi phỏng vấn.

Việc phân tách rõ ràng giữa tầng truy xuất (retrieval) và tầng suy luận (reasoning) cho phép khai thác hiệu quả năng lực của LLMs, đồng thời góp phần hạn chế hiện tượng sinh thông tin không có cơ sở (hallucination) thông qua việc kiểm soát ngữ cảnh đầu vào.

1.2.2. Kiến trúc Retrieval-Augmented Generation (RAG)

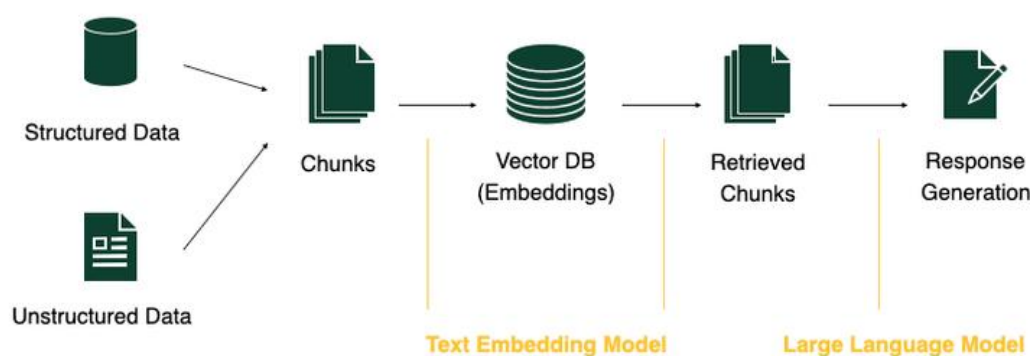
Retrieval-Augmented Generation (RAG) là kiến trúc kết hợp giữa truy xuất thông tin và sinh ngôn ngữ tự nhiên bằng mô hình ngôn ngữ lớn (Lewis và cộng sự, 2020). Trong kiến trúc này, hệ thống truy xuất các tài liệu liên quan từ nguồn dữ liệu bên ngoài và sử dụng chúng như ngữ cảnh điều kiện (conditioning context) cho quá trình sinh văn bản.

Quy trình RAG thường bao gồm ba giai đoạn chính: (i) Indexing: dữ liệu được phân đoạn (chunking), chuyển thành vector thông qua mô hình embedding và lưu trữ trong cơ sở dữ liệu vector; (ii) Retrieval: hệ thống tìm kiếm các đoạn văn bản có độ tương đồng cao nhất với truy vấn đầu vào; (iii) Generation: các đoạn văn bản được truy xuất được đưa vào prompt, giúp mô hình tổng hợp câu trả lời dựa trên thông tin liên quan.

Hiệu quả của RAG phụ thuộc trực tiếp vào chất lượng của tầng truy xuất và chiến lược phân đoạn văn bản, nhằm đảm bảo tính toàn vẹn ngữ nghĩa của thông tin. Trong bài toán đối sánh CV-JD, các phương pháp phân đoạn theo cấu trúc được ưu tiên để tách biệt rõ các nhóm thông tin như kỹ năng, kinh nghiệm và học vấn.

Bằng cách ràng buộc quá trình sinh với ngữ cảnh được truy xuất, RAG giúp giảm hiện tượng sinh thông tin không có cơ sở (hallucination) và nâng cao độ tin cậy của hệ thống. Trong phạm vi đề tài, RAG đóng vai trò như một tầng trung gian giữa truy xuất và suy luận, cung cấp ngữ cảnh có kiểm soát cho mô hình ngôn ngữ, từ đó đảm bảo các phân tích đánh giá đều có thể truy vết dựa trên dữ liệu thực tế.

Simple RAG



Hình 1.1. Minh họa kiến trúc cơ bản của hệ thống RAG

Nguồn: Chaoyu Yang, 2024

1.2.3. Hệ thống Truy xuất thông tin (IR)

a. Truy xuất từ khóa (Lexical Retrieval) với BM25

BM25, hay Okapi BM25, là một thuật toán xếp hạng tài liệu dựa trên khung mô hình liên quan xác suất (Probabilistic Relevance Framework), được sử dụng để ước lượng mức độ liên quan giữa truy vấn và tài liệu thông qua tần suất xuất hiện của từ khóa và độ dài tài liệu (Robertson & Zaragoza, 2009). Nhờ tính xác định (deterministic) cao và hiệu quả tính toán tốt, BM25 được sử dụng rộng rãi trong các hệ thống truy xuất quy mô lớn.

Cho truy vấn $q = \{t_1, t_2, \dots, t_m\}$ và tài liệu d , điểm BM25 được tính như sau:

$$score_{BM25}(q, d) = \sum_{j=1}^m IDF(t_j) \cdot \frac{f(t_j, d) \cdot (k_1 + 1)}{f(t_j, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{avgdl}\right)}$$

$$IDF(t_j) = \log \left(\frac{N - n(t_j) + 0.5}{n(t_j) + 0.5} + 1 \right)$$

trong đó, $f(t_j, d)$ là tần suất xuất hiện của thuật ngữ t_j trong tài liệu d , $|d|$ là độ dài tài liệu, $avgdl$ là độ dài trung bình của toàn bộ tập tài liệu, và k_1, b là các hyperparameter điều khiển độ bão hòa tần suất và chuẩn hóa độ dài tài liệu, N là tổng số tài liệu trong tập dữ liệu (corpus), và $n(t_j)$ là số tài liệu chứa thuật ngữ t_j . Thành phần $IDF(t_j)$ được tính nhằm giảm trọng số của các từ phổ biến và tăng trọng số cho các từ hiếm mang nhiều thông tin.

Trong các bộ dữ liệu mang tính đặc thù chuyên ngành (domain-specific corpus) chứa nhiều thuật ngữ kỹ thuật hiếm, các phương pháp khớp chính xác (exact match) như BM25 thường thể hiện ưu thế so với các mô hình embedding tổng quát. Điều này là do BM25 có khả năng tận dụng trực tiếp các tín hiệu từ khóa mang tính định danh cao, trong khi các mô hình embedding chưa được huấn luyện chuyên biệt có thể không biểu diễn chính xác các thuật ngữ kỹ thuật hiếm.

Trong bối cảnh bài toán tuyển dụng, điều này đặc biệt quan trọng khi hệ thống cần xử lý các truy vấn chứa các kỹ năng hoặc công nghệ cụ thể (ví dụ: tên ngôn ngữ lập trình, framework hoặc công cụ chuyên môn) xuất hiện trong hồ sơ ứng viên và mô tả công việc. Tuy nhiên, hạn chế của BM25 là không nắm bắt được quan hệ ngữ nghĩa giữa các từ, dẫn đến việc các biểu đạt tương đương về mặt ý nghĩa có thể nhận điểm số thấp nếu không trùng khớp từ vựng.

b. Truy xuất ngữ nghĩa (Dense Retrieval)

Dense Retrieval sử dụng các mô hình embedding để biểu diễn truy vấn q và tài liệu d dưới dạng các vector dày đặc (dense vector) trong không gian nhiều chiều.

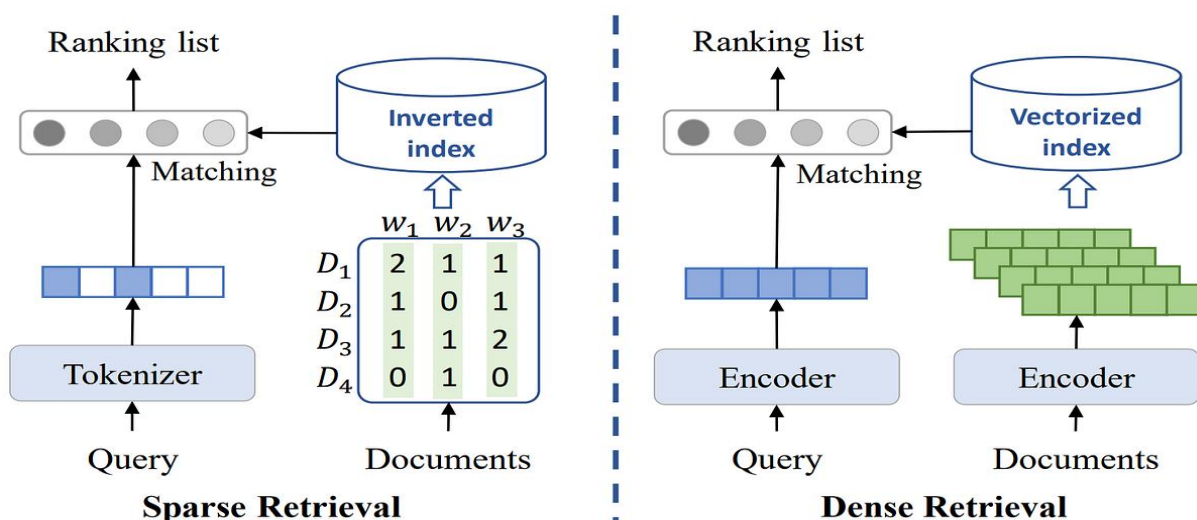
Độ tương đồng giữa truy vấn và tài liệu thường được đo bằng độ tương đồng cosine (cosine similarity), cho phép đánh giá mức độ liên quan dựa trên hướng của các vector trong không gian embedding:

$$\text{score}(q, d) = \cos(\text{emb}_q, \text{emb}_d) = \frac{\text{emb}_q \cdot \text{emb}_d}{\|\text{emb}_q\| \cdot \|\text{emb}_d\|}$$

trong đó, các vector embedding thường được chuẩn hóa về độ dài đơn vị, khi đó cosine similarity tương đương với tích vô hướng.

Trong các hệ thống hiện đại, phương pháp truy xuất bất đối xứng (asymmetric retrieval) thường được sử dụng, trong đó truy vấn và tài liệu được mã hóa theo các chiến lược khác nhau nhằm phản ánh sự khác biệt về độ dài và vai trò ngữ nghĩa. Cách tiếp cận này đặc biệt hiệu quả trong các bài toán mà truy vấn thường ngắn và mang tính khái quát, trong khi tài liệu có cấu trúc dài và chi tiết.

Quá trình truy xuất thường được tối ưu hóa thông qua các cấu trúc chỉ mục vector chuyên dụng như FAISS (Johnson và cộng sự, 2021), cho phép thực hiện tìm kiếm tương đồng hiệu quả trên không gian vector có kích thước lớn. Nhờ khả năng biểu diễn ngữ nghĩa, Dense Retrieval giúp giải quyết bài toán bất đồng từ vựng (vocabulary mismatch), vốn là hạn chế của các phương pháp truy xuất dựa trên từ khóa (Karpukhin và cộng sự, 2020).



Hình 1.2. So sánh kiến trúc hệ thống Truy xuất thưa (Sparse Retrieval) dựa trên từ khóa và Truy xuất ngữ nghĩa (Dense Retrieval) dựa trên vector

Nguồn: Dựa theo Zhou và cộng sự, 2022

Trong bối cảnh bài toán tuyển dụng, các kỹ năng hoặc kinh nghiệm có thể được diễn đạt bằng nhiều cách khác nhau giữa CV và JD. Dense Retrieval cho phép hệ thống nhận diện các biểu đạt tương đương về mặt ý nghĩa, từ đó cải thiện khả năng phát hiện các ứng viên tiềm năng ngay cả khi không có sự trùng khớp trực tiếp về từ vựng. Tuy nhiên, các mô hình embedding tổng quát có thể gặp hạn chế trong việc biểu diễn chính xác các thuật ngữ kỹ thuật hiếm trong domain chuyên biệt. Điều này có thể dẫn đến việc đánh giá sai mức độ liên quan trong các bài toán yêu cầu độ chính xác cao theo từ khóa, chẳng hạn như tuyển dụng kỹ thuật.

c. Kỹ thuật kết hợp Reciprocal Rank Fusion (RRF)

Reciprocal Rank Fusion (RRF) là phương pháp kết hợp các danh sách xếp hạng từ nhiều hệ thống truy xuất khác nhau mà không yêu cầu chuẩn hóa điểm số hoặc huấn luyện thêm (Cormack và cộng sự, 2009). Phương pháp này đặc biệt hiệu quả khi các hệ thống thành phần có đặc tính phân phối điểm số khác nhau, như giữa truy xuất từ khóa (BM25) và truy xuất ngữ nghĩa (Dense Retrieval).

Cho một tập các hệ thống truy xuất, điểm RRF của một tài liệu d được tính như sau:

$$score_{RRF}(d) = \sum_{i=1}^N \frac{1}{k + rank_i(d)}$$

trong đó $rank_i(d)$ là thứ hạng của tài liệu d trong hệ thống thứ i , và k là hằng số làm mượt (smoothing constant). Giá trị của k giúp giảm ảnh hưởng của các tài liệu có thứ hạng thấp, đồng thời đảm bảo tính ổn định của phép kết hợp.

Ưu điểm nổi bật của RRF là khả năng kết hợp hiệu quả các tín hiệu truy xuất không đồng nhất. Trong kiến trúc hybrid retrieval, RRF cho phép tận dụng đồng thời khả năng khớp chính xác từ khóa của BM25 và khả năng nắm bắt ngữ nghĩa của Dense Retrieval, từ đó tạo ra một danh sách ứng viên có chất lượng cao hơn so với từng phương pháp riêng lẻ.

Trong bối cảnh bài toán đối sánh CV-JD, dữ liệu thường chứa đồng thời các tín hiệu từ khóa mang tính định danh cao và các quan hệ ngữ nghĩa tiềm ẩn. Việc sử dụng RRF giúp hệ thống khai thác hiệu quả cả hai loại tín hiệu này, từ đó cải thiện khả năng phát hiện ứng viên phù hợp trong giai đoạn truy xuất ban đầu.

Tuy nhiên, RRF cũng có một số hạn chế, đáng chú ý là giả định ngầm rằng các hệ thống thành phần đóng góp ngang nhau, trong khi trên thực tế mức độ quan trọng của từng tín hiệu có thể khác nhau và cần được đánh giá thông qua thực nghiệm.

1.2.4. Phương pháp Learning-to-Rank (LTR)

a. Khái niệm LTR

Learning-to-Rank (LTR) là hướng tiếp cận sử dụng học máy (machine learning) để xây dựng các mô hình xếp hạng tự động, với mục tiêu học một hàm xếp hạng $f: Q \times D \rightarrow \mathbb{R}$ nhằm sắp xếp các tài liệu d theo mức độ liên quan đối với truy vấn q (Liu, 2009). Khác với các bài toán phân loại hoặc hồi quy độc lập, LTR tập trung tối ưu hóa thứ tự tương đối giữa các tài liệu trong cùng một truy vấn, thường dựa trên các độ đo đánh giá xếp hạng như NDCG.

Đầu vào của mô hình LTR là các feature vector $\phi(q, d) \in \mathbb{R}^k$ biểu diễn mối quan hệ giữa truy vấn và tài liệu. Các đặc trưng này có thể bao gồm tín hiệu truy xuất (retrieval signals), mức độ phù hợp kỹ năng, và các đặc trưng cấu trúc của tài liệu.

Trong phạm vi đề tài, LTR đóng vai trò là thành phần cốt lõi trong tầng xếp hạng, cho phép kết hợp nhiều tín hiệu khác nhau để xác định thứ tự ưu tiên giữa các ứng viên. Cách tiếp cận này phù hợp với bản chất của bài toán tuyển dụng, nơi mục tiêu không phải là phân loại độc lập từng ứng viên, mà là xây dựng một danh sách xếp hạng tối ưu nhằm hỗ trợ ra quyết định.

b. Thuật toán LambdaMART

LambdaMART là một thuật toán Learning-to-Rank dựa trên mô hình cây quyết định tăng cường (Gradient Boosted Decision Trees), được thiết kế nhằm tối ưu trực tiếp chất lượng xếp hạng. Khác với các phương pháp hồi quy truyền thống tối ưu sai số dự đoán điểm số, LambdaMART hướng tới việc tối ưu các độ đo đánh giá xếp hạng như NDCG.

Ý tưởng cốt lõi của LambdaMART là sử dụng gradient boosting kết hợp với cơ chế lambda gradients để xấp xỉ đạo hàm của các độ đo xếp hạng không khả vi. Thay vì tối ưu trực tiếp giá trị điểm số, mô hình học cách điều chỉnh thứ tự tương đối giữa các tài liệu nhằm cải thiện thứ hạng tổng thể (Burges, 2010).

Trong phân loại các phương pháp Learning-to-Rank, LambdaMART thường được xem là phương pháp thuộc nhóm pairwise, trong đó mô hình học cách so sánh thứ tự giữa các cặp tài liệu dựa trên mức độ liên quan tương đối. Tuy nhiên, nhờ việc tối ưu

trực tiếp theo các độ đo như NDCG, phương pháp này cũng thể hiện hiệu quả gần với các cách tiếp cận listwise trong thực tế.

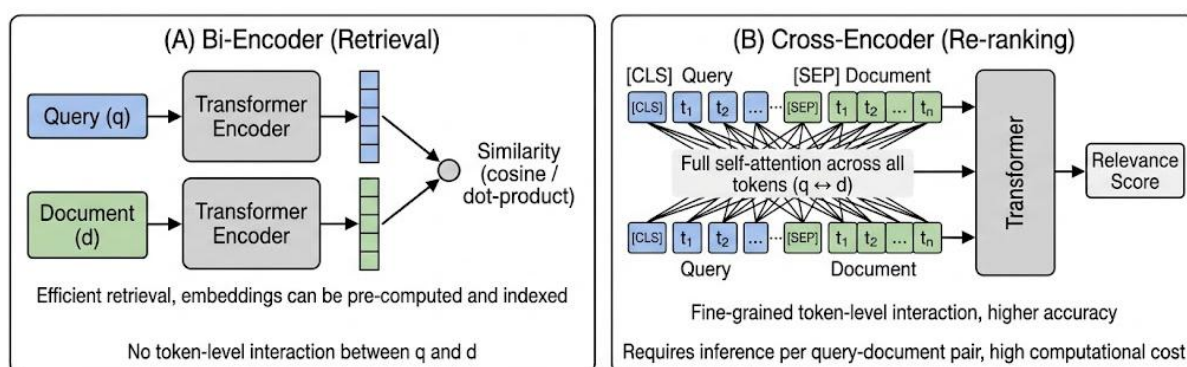
Đầu vào của LambdaMART là các feature vector $\phi(q, d) \in \mathbb{R}^k$ biểu diễn mối quan hệ giữa truy vấn và tài liệu. Trong quá trình huấn luyện, mô hình tự động học được tầm quan trọng của các đặc trưng này thông qua cơ chế boosting, từ đó kết hợp nhiều tín hiệu khác nhau để đưa ra điểm số xếp hạng.

Trong phạm vi đề tài, LambdaMART được sử dụng như một thành phần học xếp hạng nhằm tích hợp các tín hiệu truy xuất và đặc trưng miền (domain-specific features), từ đó cải thiện thứ tự ưu tiên giữa các ứng viên so với các phương pháp chấm điểm thủ công.

c. Kỹ thuật xếp hạng lại (re-ranking) với Cross-Encoder

Mặc dù các phương pháp truy xuất ở giai đoạn đầu như BM25 và Bi-Encoder (Dense Retrieval) đạt hiệu quả cao về tốc độ, chúng vẫn bị hạn chế trong việc nắm bắt các tương tác ngữ nghĩa chi tiết giữa truy vấn và tài liệu. Cụ thể, trong kiến trúc Bi-Encoder, truy vấn và tài liệu được mã hóa độc lập, do đó không thể mô hình hóa trực tiếp các mối quan hệ ở cấp độ token giữa hai văn bản.

Để khắc phục hạn chế này, kiến trúc Cross-Encoder được sử dụng như một bước xếp hạng lại (re-ranking) ở giai đoạn cuối nhằm tăng độ chính xác (Nogueira & Cho, 2019). Khác với Bi-Encoder, Cross-Encoder nhận đầu vào là cặp văn bản kết hợp giữa truy vấn q và tài liệu d , cho phép cơ chế self-attention trong Transformer tính toán trực tiếp tương tác giữa mọi token của hai văn bản. Nhờ đó, mô hình có thể nắm bắt các tín hiệu ngữ nghĩa tinh vi, từ đó đánh giá chính xác hơn mức độ liên quan.



Hình 1.3. So sánh kiến trúc Bi-Encoder và Cross-Encoder trong bài toán truy xuất

Nguồn: Tác giả tự tổng hợp

Tuy nhiên, sự gia tăng về độ chính xác thường đi kèm với chi phí tính toán đáng kể. Do không thể tính toán trước và lưu trữ biểu diễn vector như Bi-Encoder, Cross-

Encoder phải thực hiện suy luận riêng cho từng cặp (q, d) , khiến phương pháp này không khả thi ở quy mô lớn (Reimers & Gurevych, 2019).

Vì vậy, kiến trúc “retrieve-then-rerank” thường được áp dụng, trong đó Cross-Encoder chỉ được sử dụng để đánh giá lại một tập con nhỏ các tài liệu đã được truy xuất từ giai đoạn đầu. Cách tiếp cận này cho phép cân bằng giữa hiệu quả và độ chính xác của hệ thống. Trong phạm vi đề tài, Cross-Encoder được sử dụng như một tầng xếp hạng lại nhằm tinh chỉnh thứ tự các ứng viên tiềm năng, góp phần cải thiện chất lượng xếp hạng cuối cùng.

1.2.5. Hệ thống Đa tác tử (Multi-Agent System)

Để giải quyết các bài toán vận hành phức tạp, các hệ thống hiện đại ngày càng chuyển dịch từ kiến trúc nguyên khối (monolithic) sang các kiến trúc phân tán, trong đó hệ thống đa tác tử (Multi-Agent Systems - MAS) là một hướng tiếp cận tiêu biểu (Wooldridge, 2009). Trong kiến trúc này, hệ thống được cấu thành từ nhiều tác tử (agents), mỗi tác tử hoạt động như một thực thể xử lý độc lập, có khả năng tự trị và thực hiện một nhiệm vụ chuyên biệt trong toàn bộ quy trình.

Đặc biệt trong bối cảnh trí tuệ nhân tạo hiện tại, sự ra đời của các hệ thống đa tác tử dựa trên mô hình ngôn ngữ lớn (LLM-based Multi-Agent Systems) đã mở ra một hướng tiếp cận mới đầy tiềm năng (Xi và cộng sự, 2023). Các tác vụ phức tạp được phân rã thành một chuỗi các bước xử lý liên kết với nhau dưới dạng đồ thị luồng công việc (agentic workflow). Trong đó, mỗi tác tử đảm nhiệm một chức năng cụ thể như trích xuất thông tin, phân tích yêu cầu hoặc tổng hợp kết quả. Việc tổ chức theo cấu trúc đồ thị định hướng cho phép kiểm soát luồng dữ liệu (data flow) và thứ tự thực thi một cách nghiêm ngặt, đồng thời hỗ trợ cơ chế điều phối (orchestration) nhịp nhàng giữa các tác tử.

Kiến trúc đa tác tử đặc biệt phù hợp trong các hệ thống đòi hỏi tích hợp nhiều phương pháp xử lý lai (hybrid pipeline). Cụ thể, nó cho phép kết hợp trộn trau giữa các thành phần chấm điểm định lượng chuẩn xác (deterministic scoring) dựa trên thuật toán (như BM25, LambdaMART) với các thành phần suy luận ngôn ngữ linh hoạt (LLM-based reasoning), từ đó tạo ra một quy trình xử lý vừa mạnh mẽ vừa minh bạch.

Trong bối cảnh bài toán tuyển dụng, quy trình đánh giá hồ sơ ứng viên bao gồm nhiều bước phức tạp như trích xuất thông tin, tiêu chuẩn hóa yêu cầu và phân tích mức độ phù hợp. Kiến trúc đa tác tử cho phép ánh xạ trực tiếp các bước này thành các tác tử

độc lập, giúp hệ thống phản ánh độ chính xác cao quy trình nghiệp vụ thực tế. Đồng thời, cách tiếp cận này đảm bảo nguyên lý thiết kế phần mềm với tính kết dính nội bộ cao (high cohesion) và giảm sự phụ thuộc giữa các thành phần (low coupling), góp phần nâng cao khả năng mở rộng (scalability) và khả năng bảo trì của toàn bộ hệ thống. Trong tổng thể hệ thống, kiến trúc đa tác tử đóng vai trò như lớp điều phối (orchestration layer), kết nối các thành phần truy xuất, xếp hạng và suy luận thành một pipeline thống nhất.

1.2.6. Các độ đo đánh giá

a. Đánh giá chất lượng xếp hạng

Khác với các bài toán phân loại truyền thống, bài toán đối sánh CV-JD mang bản chất là bài toán xếp hạng, trong đó thứ tự các kết quả đóng vai trò quyết định. Trong thực tế, nhà tuyển dụng chủ yếu quan tâm đến nhóm ứng viên đứng đầu danh sách, do đó các độ đo đánh giá cần phản ánh chính xác chất lượng xếp hạng ở các vị trí đầu. Các độ đo được sử dụng trong nghiên cứu bao gồm:

- **NDCG (Normalized Discounted Cumulative Gain)** (Järvelin & Kekäläinen, 2002): Là độ đo đánh giá chất lượng xếp hạng có xét đến vị trí của tài liệu trong danh sách. NDCG áp dụng cơ chế giảm trọng số theo vị trí, qua đó ưu tiên các kết quả liên quan xuất hiện ở top đầu. Đây là độ đo cốt lõi trong các bài toán Learning-to-Rank và phản ánh sát hành vi thực tế trong tuyển dụng.
- **MAP (Mean Average Precision)** (Manning và cộng sự, 2008): Là trung bình của Average Precision trên toàn bộ tập truy vấn, phản ánh mức độ chính xác tích lũy của hệ thống. Chỉ số này đánh giá khả năng duy trì chất lượng xếp hạng ổn định trên nhiều mô tả công việc khác nhau.
- **MRR (Mean Reciprocal Rank)** (Manning và cộng sự, 2008): Đo lường vị trí xuất hiện của kết quả phù hợp đầu tiên trong danh sách. MRR đặc biệt phù hợp trong các kịch bản yêu cầu đưa ra ứng viên tốt nhất ở vị trí đầu, giúp đánh giá khả năng ra quyết định nhanh của hệ thống.

b. Đánh giá chất lượng sinh văn bản với RAGAS

Trong các mô-đun sử dụng LLM để sinh nhận xét hoặc câu hỏi phỏng vấn, việc đánh giá chất lượng đầu ra không thể dựa trên các độ đo so khớp từ vựng truyền thống như BLEU (Papineni và cộng sự, 2002) hay ROUGE (Lin, 2004), do không tồn tại một ground truth cố định. Thay vào đó, framework RAGAS (Es và cộng sự, 2024) cung cấp

một cách tiếp cận đánh giá tự động đa chiều, tập trung vào tính đúng đắn và độ tin cậy của nội dung. Các độ đo chính bao gồm:

- **Faithfulness:** Đánh giá mức độ các thông tin trong câu trả lời có thể được truy vết và hỗ trợ hoàn toàn bởi ngữ cảnh đầu vào (context). Đây là tiêu chí sống còn trong tuyển dụng, đảm bảo mọi nhận xét về ứng viên đều dựa trên dữ liệu thực tế từ CV, giảm thiểu hiện tượng ảo giác của LLM.
- **Context Precision:** Đo lường tỷ lệ thông tin hữu ích thực sự có trong ngữ cảnh được truy xuất. Giá trị thấp cho thấy hệ thống RAG đang nhồi nhét quá nhiều thông tin nhiễu, làm giảm hiệu năng suy luận của mô hình ngôn ngữ.
- **Context Recall:** Đánh giá xem hệ thống truy xuất đã gom đủ các thông tin cần thiết để trả lời trọn vẹn truy vấn hay chưa. Giá trị thấp phản ánh sự thiếu sót dữ liệu nghiêm trọng ở tầng truy xuất.
- **Answer Relevancy:** Đo lường mức độ đầu ra thực sự giải quyết trực tiếp yêu cầu của người dùng, đảm bảo LLM không trả lời vòng vo hoặc lệch khỏi chủ đề của JD.

Việc kết hợp hệ thống độ đo RAGAS với các metric xếp hạng truyền thống (NDCG, MAP) cho phép đánh giá toàn diện cả hai khía cạnh cốt lõi của TalentOS: Độ chính xác tuyệt đối trong việc sàng lọc ứng viên và sự tin cậy trong các phân tích ngôn ngữ tự nhiên.

CHƯƠNG 2: PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG

2.1. Yêu cầu hệ thống

Như đã trình bày tại Chương 1, bài toán đối sánh giữa CV và JD đã được tái định nghĩa dưới góc nhìn của bài toán truy xuất và xếp hạng tài liệu. Tuy nhiên, để hiện thực hóa mô hình lý thuyết này thành một hệ thống phần mềm có khả năng vận hành trong môi trường thực tế, cần tiến hành phân tích các yêu cầu hệ thống dưới cả hai khía cạnh: chức năng (functional requirements) và phi chức năng (non-functional requirements).

Việc xác định rõ các yêu cầu này đóng vai trò nền tảng trong thiết kế kiến trúc pipeline của hệ thống TalentOS, đảm bảo sự cân bằng giữa độ chính xác của thuật toán, hiệu năng xử lý và khả năng triển khai thực tiễn.

2.1.1. Yêu cầu chức năng

Hệ thống được thiết kế nhằm phục vụ hai nhóm người dùng chính trong quy trình tuyển dụng:

- **Chuyên viên tuyển dụng (HR):** cần đánh giá nhanh mức độ phù hợp của ứng viên đối với một vị trí cụ thể.
- **Quản lý tuyển dụng:** cần so sánh và ra quyết định dựa trên nhiều hồ sơ ứng viên trong cùng một đợt tuyển dụng.

Từ góc độ này, hệ thống phải cung cấp một tập các chức năng cốt lõi phản ánh trực tiếp pipeline xử lý dữ liệu từ truy xuất, xếp hạng đến sinh phân tích. Cụ thể:

- **[FR-01] Phân tích CV đơn lẻ:**

Cho phép người dùng tải lên một hồ sơ ứng viên (PDF/DOC/DOCX) cùng với JD dạng văn bản. Hệ thống thực hiện toàn bộ pipeline và trả về:

- Điểm phù hợp tổng thể (overall score)
- Điểm theo từng nhóm tiêu chí
- Danh sách kỹ năng khớp và kỹ năng thiếu
- Nhận xét tự động (AI verdict)
- Bộ câu hỏi phỏng vấn gợi ý

- **[FR-02] Phân tích hàng loạt (Batch processing):**

Cho phép tải lên và xử lý đồng thời một tập hồ sơ ứng viên đối với cùng một mô tả công việc. Kích thước mỗi lô xử lý được kiểm soát thông qua tham số

batch_size (cấu hình mặc định là 20 hồ sơ/lô), nhằm đảm bảo cân bằng giữa hiệu năng hệ thống, giới hạn tần suất gọi API (rate limits) từ các dịch vụ LLM và trải nghiệm người dùng trong quá trình xử lý. Khi thực thi, hệ thống thực hiện:

- Truy xuất và tái xếp hạng (retrieval và re-ranking) trên toàn tập ứng viên
- Trả về danh sách ứng viên đã được sắp xếp theo thứ tự giảm dần của điểm đánh giá tổng thể (overall score)
- **[FR-03] Phân tích theo luồng (Streaming):** Mở rộng từ batch processing bằng cách trả kết quả theo thời gian thực thông qua cơ chế Server-Sent Events (SSE), cho phép hiển thị từng kết quả ngay khi hoàn thành, cải thiện trải nghiệm người dùng.
- **[FR-04] Quản lý lịch sử phân tích:**
Lưu trữ và truy vấn các phiên phân tích đã thực hiện, hỗ trợ:
 - Lọc theo recommendation (shortlist/hold/reject)
 - Sắp xếp theo thời gian hoặc score
 - Phân trang dữ liệu
- **[FR-05] Xem chi tiết và cập nhật quyết định:** Cho phép truy xuất đầy đủ kết quả phân tích của từng phiên và ghi nhận quyết định cuối cùng của người dùng (accept/reject), tách biệt với recommendation do hệ thống đề xuất.

Các chức năng cốt lõi (phân tích đơn lẻ, batch và streaming) phản ánh trực tiếp kiến trúc pipeline của hệ thống theo mô hình: **Retrieval** → **Re-ranking** → **Generation**, trong đó mỗi giai đoạn tương ứng với một lớp xử lý độc lập, đảm bảo tính module (modularity) và khả năng mở rộng của hệ thống.

2.1.2. Yêu cầu phi chức năng

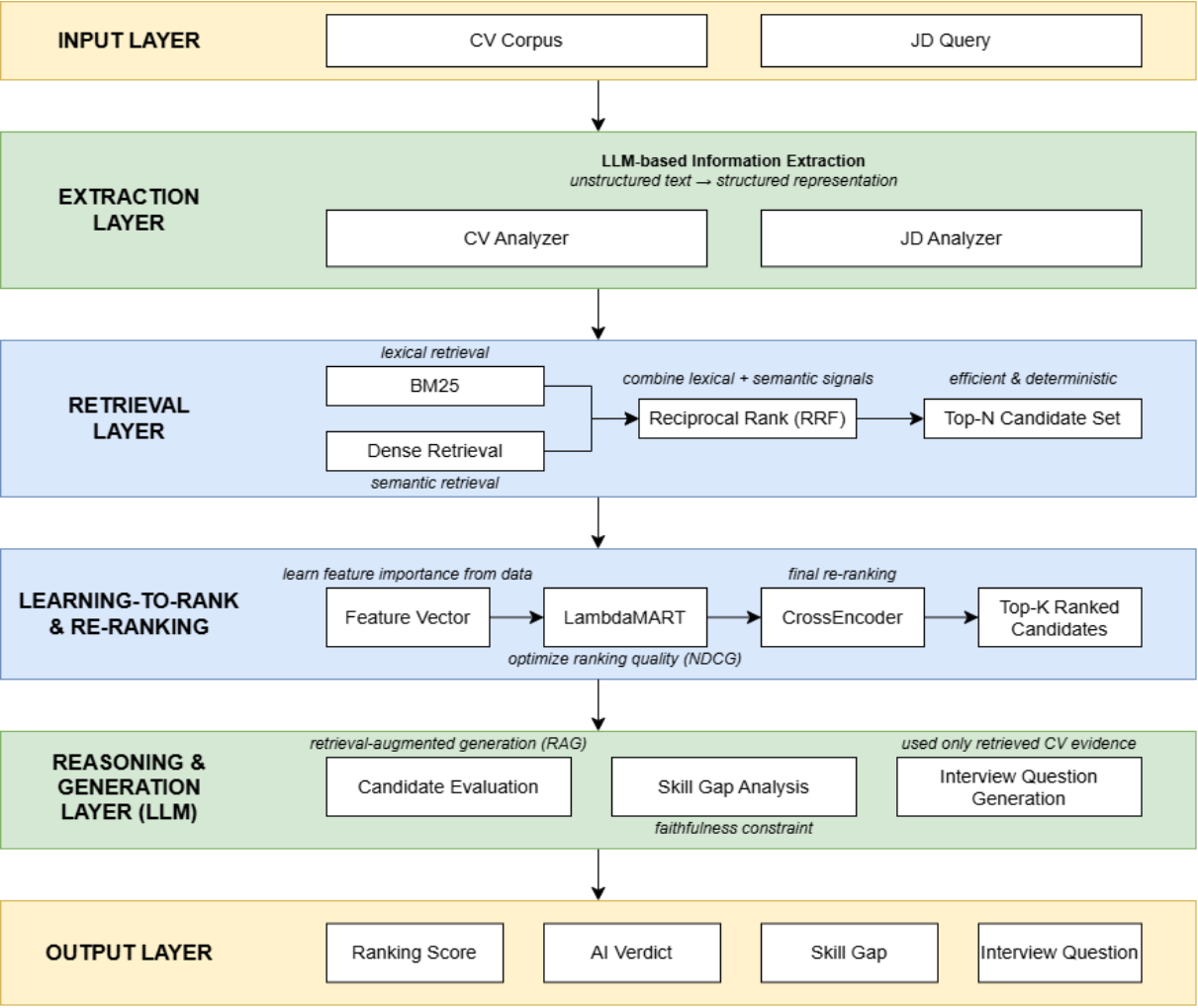
Bên cạnh các chức năng nghiệp vụ, hệ thống cần đáp ứng các ràng buộc phi chức năng nhằm đảm bảo khả năng vận hành ổn định và hiệu quả trong môi trường thực tế:

- **[NFR-01] Hiệu năng (Performance):** Thời gian xử lý cho mỗi hồ sơ được thiết kế nhằm nằm trong ngưỡng chấp nhận được trong thực tế (thông thường dưới 30 giây trong điều kiện vận hành tiêu chuẩn). Trong chế độ batch, hệ thống hỗ trợ xử lý song song với cơ chế giới hạn số lượng tác vụ đồng thời nhằm kiểm soát tài nguyên và đảm bảo tính ổn định của hệ thống.

- **[NFR-02] Khả năng mở rộng (Scalability):** Hệ thống được thiết kế theo kiến trúc pipeline nhiều tầng, cho phép thay thế độc lập các thành phần như retrieval model, ranking model hoặc LLM mà không ảnh hưởng toàn cục.
- **[NFR-03] Độ tin cậy (Reliability):** Hệ thống cần xử lý ổn định các lỗi từ dịch vụ bên ngoài (LLM API), bao gồm cơ chế retry và backoff nhằm đảm bảo quá trình xử lý không bị gián đoạn.
- **[NFR-04] Kiểm soát hallucination (Faithfulness constraint):** Các nội dung sinh bởi LLM phải được ràng buộc chặt chẽ bởi dữ liệu truy xuất, đảm bảo mọi nhận xét đều có thể truy vết từ CV và JD đầu vào.
- **[NFR-05] Khả năng triển khai (Deployability):** Hệ thống phải được đóng gói bằng Docker và hỗ trợ triển khai nhanh trong môi trường thực tế mà không yêu cầu hạ tầng tính toán chuyên biệt.
- **[NFR-06] Bảo mật và riêng tư dữ liệu (Data Privacy):** Không lưu trữ file CV thô trên hệ thống, chỉ xử lý trong bộ nhớ và lưu trữ kết quả đã được chuẩn hóa nhằm đảm bảo an toàn dữ liệu cá nhân.

Tóm lại, tập hợp các yêu cầu chức năng – đặc biệt là [FR-02] (xử lý hàng loạt) và [FR-03] (xử lý theo luồng) – cùng với các ràng buộc nghiêm ngặt về hiệu năng [NFR-01] và tính trung thực [NFR-04] đã đặt ra một bài toán tối ưu đa mục tiêu phức tạp. Cụ thể, hệ thống phải đối mặt với một sự đánh đổi nội tại giữa hai mục tiêu: (i) tối đa hóa độ chính xác và khả năng hiểu ngữ nghĩa thông qua các mô hình học sâu, và (ii) đảm bảo thời gian xử lý đáp ứng các yêu cầu thực tiễn. Sự mâu thuẫn này cho thấy việc sử dụng một mô hình đơn lẻ theo cách tiếp cận toàn diện (brute-force) là không khả thi trong bối cảnh hệ thống. Thay vào đó, hệ thống cần được thiết kế dưới dạng một pipeline phân tầng, trong đó mỗi thành phần đảm nhiệm một vai trò cụ thể nhằm cân bằng giữa hiệu năng và chất lượng đầu ra. Các quyết định thiết kế thuật toán nhằm giải quyết bài toán đánh đổi này sẽ được mô hình hóa và phân tích chi tiết trong Chương 3.

2.2. Kiến trúc tổng thể và luồng xử lý thuật toán



Hình 2.1. Kiến trúc tổng thể và luồng xử lý thuật toán của hệ thống TalentOS

Nguồn: Tác giả tự tổng hợp

Để đáp ứng các yêu cầu hệ thống đã được xác định, hệ thống TalentOS được thiết kế theo kiến trúc pipeline nhiều giai đoạn (multi-stage pipeline), kết hợp giữa các phương pháp truy xuất thông tin truyền thống, mô hình Learning-to-Rank và LLM. Kiến trúc này cho phép phân tách rõ ràng giữa các thành phần truy xuất, xếp hạng và suy luận, từ đó tối ưu đồng thời hiệu năng xử lý và chất lượng kết quả.

Như được thể hiện trong Hình 2.1, hệ thống được tổ chức thành các tầng xử lý nối tiếp nhau, trong đó mỗi tầng đảm nhiệm một chức năng cụ thể và truyền kết quả đã được xử lý sang tầng tiếp theo. Luồng dữ liệu đi từ đầu vào (CV và JD) đến đầu ra (kết quả xếp hạng và phân tích), thông qua các bước trích xuất thông tin, truy xuất, xếp hạng và sinh phân tích.

a. Tầng đầu vào (Input layer)

Tầng đầu vào tiếp nhận hai nguồn dữ liệu chính bao gồm tập hợp các hồ sơ ứng viên (CV Corpus) dưới dạng văn bản tự nhiên (định dạng PDF/DOC/DOCX) và mô tả công việc (JD Query) dưới dạng văn bản. Hai thành phần này tương ứng với cặp tài liệu và truy vấn (document, query) trong bài toán truy xuất thông tin đã được trình bày ở Chương 1.

b. Tầng trích xuất (Extraction layer)

Tầng trích xuất đảm nhiệm việc chuyển đổi dữ liệu phi cấu trúc thành dạng biểu diễn có cấu trúc thông qua LLM.

Cụ thể, tác tử CV Analyzer chịu trách nhiệm trích xuất các thông tin quan trọng từ hồ sơ ứng viên, bao gồm kỹ năng, kinh nghiệm, học vấn và thâm niên. Tương tự, tác tử JD Analyzer thực hiện chuẩn hóa các yêu cầu tuyển dụng từ mô tả công việc. Kết quả của bước này là các biểu diễn có cấu trúc, giúp giảm nhiễu từ văn bản thô và thiết lập nền tảng dữ liệu nhất quán cho các bước xử lý tiếp theo.

c. Tầng truy xuất (Retrieval layer)

Tầng truy xuất thực hiện bước lọc sơ bộ nhằm xác định các ứng viên có mức độ liên quan cao đối với mô tả công việc.

Hệ thống triển khai cơ chế hybrid retrieval, kết hợp giữa lexical signals và semantic signals. Cụ thể, phương pháp BM25 được sử dụng để khai thác khả năng khớp từ khóa chính xác, đặc biệt hiệu quả với các thuật ngữ kỹ thuật, trong khi các mô hình embedding được sử dụng để đo lường mức độ tương đồng ngữ nghĩa giữa CV và JD.

Hai danh sách kết quả này được kết hợp thông qua phương pháp RRF, từ đó tạo ra một tập ứng viên tiềm năng để chuyển sang giai đoạn xếp hạng chuyên sâu. Trong thiết kế của hệ thống, bước truy xuất không chỉ nhằm giảm không gian tìm kiếm, mà còn đóng vai trò ưu tiên và tái tổ chức các ứng viên tiềm năng, tạo đầu vào phù hợp cho các bước xử lý chi phí cao hơn như re-ranking và generation.

d. Tầng học xếp hạng và tái xếp hạng (Learning-to-Rank & Re-ranking Layer)

Tầng này tập trung tinh chỉnh thứ hạng của các ứng viên tiềm năng thông qua việc đánh giá đa chiều.

Hệ thống xây dựng các feature vector tổng hợp từ nhiều nguồn tín hiệu khác nhau, bao gồm thông tin truy xuất, kỹ năng và cấu trúc hồ sơ. Các đặc trưng này được

đưa vào mô hình LambdaMART nhằm tối ưu hóa chất lượng xếp hạng theo độ đo NDCG.

Sau đó, mô hình Cross-Encoder được áp dụng để mô hình hóa tương tác trực tiếp giữa CV và JD, qua đó cải thiện độ chính xác ở mức độ chi tiết. Do chi phí tính toán cao, Cross-Encoder được triển khai theo chiến lược **coarse-to-fine ranking**, chỉ áp dụng trên tập ứng viên đã được lọc từ các tầng trước, thay vì toàn bộ dữ liệu ban đầu. Thiết kế này cho phép hệ thống cân bằng hiệu quả giữa chất lượng xếp hạng và chi phí tính toán.

e. Tầng suy luận và sinh ngôn ngữ (Reasoning & Generation Layer)

Sau khi danh sách ứng viên đã được xếp hạng, tầng này sử dụng kiến trúc RAG để sinh các phân tích có ý nghĩa thực tiễn.

Các tác vụ chính bao gồm đánh giá mức độ phù hợp của ứng viên, phân tích khoảng trống kỹ năng và sinh câu hỏi phỏng vấn. Để đảm bảo tính trung thực của thông tin, quá trình sinh được ràng buộc bởi dữ liệu đã được trích xuất và xử lý ở các tầng trước, hạn chế việc suy luận ngoài ngữ cảnh.

f. Tầng đầu ra (Output Layer)

Tầng đầu ra tổng hợp kết quả từ toàn bộ pipeline và trả về dưới dạng báo cáo hoàn chỉnh cho người dùng. Kết quả bao gồm điểm đánh giá tổng thể, điểm theo từng tiêu chí, danh sách kỹ năng phù hợp và còn thiếu, nhận xét tự động từ hệ thống, cũng như bộ câu hỏi phỏng vấn gợi ý. Việc kết hợp giữa thông tin định lượng và định tính giúp hỗ trợ hiệu quả cho quá trình ra quyết định trong tuyển dụng.

2.3. Cơ chế phối hợp và giao thức giao tiếp Đa tác tử

Dựa trên kiến trúc đã trình bày tại Mục 2.2., hệ thống TalentOS được phân rã thành nhiều tác tử chuyên biệt nhằm xử lý từng giai đoạn trong pipeline. Tuy nhiên, việc phân rã này đặt ra một thách thức quan trọng: làm thế nào để đảm bảo luồng dữ liệu giữa các tác tử được truyền tải một cách nhất quán, chính xác và không làm phát sinh sai lệch ngữ nghĩa.

Để giải quyết vấn đề này, hệ thống thiết kế một cơ chế phối hợp dựa trên giao thức dữ liệu có cấu trúc (data contract), trong đó mọi thông tin trao đổi giữa các tác tử đều được chuẩn hóa dưới định dạng JSON thay vì văn bản tự nhiên.

2.3.1. Chuẩn hóa dữ liệu đầu ra

Tại tầng trích xuất, các tác tử như CV Analyzer và JD Analyzer không tạo ra văn bản tự do mà bị ràng buộc bởi một cấu trúc dữ liệu định trước. Thông tin được ánh xạ vào các trường cụ thể như danh sách kỹ năng, số năm kinh nghiệm, cấp bậc hoặc các thuộc tính liên quan khác.

Cách tiếp cận này giúp chuyển đổi đầu ra của mô hình ngôn ngữ – vốn mang tính xác suất và phi cấu trúc – thành dữ liệu có tính xác định và có thể xử lý trực tiếp bởi các thuật toán ở các tầng tiếp theo, bao gồm retrieval và learning-to-rank.

2.3.2. Cơ chế luân chuyển và lắp ghép ngữ cảnh

Trong giai đoạn suy luận, các tác tử không làm việc độc lập mà nhận đầu vào là sự kết hợp của nhiều nguồn dữ liệu đã được xử lý trước đó. Cụ thể, đầu vào của tầng suy luận bao gồm:

- Dữ liệu có cấu trúc được trích xuất từ CV và JD
- Các tín hiệu định lượng từ các tầng trước

Các thành phần này được hệ thống điều phối (orchestrator) tổng hợp thành một ngữ cảnh đầu vào thống nhất cho mô hình ngôn ngữ. Việc sử dụng dữ liệu đã được chuẩn hóa thay vì văn bản thô giúp giảm đáng kể độ nhiễu và tăng tính nhất quán trong quá trình suy luận.

2.3.3. Ràng buộc suy luận

Một nguyên tắc thiết kế quan trọng của hệ thống là các tác tử suy luận không được phép truy cập trực tiếp vào văn bản CV thô. Thay vào đó, chúng chỉ được sử dụng dữ liệu đã được trích xuất và chuẩn hóa từ các tầng trước. Ràng buộc này giúp:

- Giới hạn không gian suy luận của mô hình
- Ngăn chặn việc sinh thông tin không có cơ sở (hallucination)
- Đảm bảo mọi kết quả đầu ra đều có thể truy vết từ dữ liệu đầu vào

Việc sử dụng giao thức dữ liệu có cấu trúc như một lớp trung gian giúp kết nối các tác tử trong hệ thống thành một pipeline thống nhất. Thiết kế này không chỉ đảm bảo tính nhất quán của dữ liệu mà còn nâng cao tính minh bạch và khả năng kiểm soát của toàn bộ hệ thống.

2.4. Chiến lược quản trị dữ liệu

Chiến lược quản trị dữ liệu đóng vai trò quan trọng trong việc đảm bảo hiệu năng và khả năng mở rộng của hệ thống, đặc biệt trong bối cảnh kết hợp giữa truy xuất thời gian thực và lưu trữ kết quả phân tích phức tạp. Trong hệ thống TalentOS, yêu cầu này dẫn đến việc lựa chọn một kiến trúc lưu trữ hai tầng (dual-layer storage strategy), phân tách rõ ràng giữa dữ liệu phục vụ tính toán thời gian thực và dữ liệu phục vụ quản lý lâu dài.

2.4.1. Tầng lưu trữ phục vụ tính toán nhanh (In-memory/File-based Storage)

Tầng này được thiết kế nhằm phục vụ trực tiếp cho tầng truy xuất, nơi yêu cầu độ trễ thấp và hiệu năng cao.

- **Chỉ mục tĩnh:** Các chỉ mục (như BM25 Index) được tuần tự hóa thành tệp nhị phân và nạp toàn bộ lên bộ nhớ chính (RAM) khi khởi động. Việc này loại bỏ hoàn toàn nút thắt cổ chai về truy xuất ổ cứng (I/O bottleneck) khi thực hiện so khớp từ vựng.
- **Giao thức dữ liệu (Data Contract):** Để tối ưu tốc độ truyền tải ngữ cảnh giữa các module, hệ thống định nghĩa một tập hợp các cấu trúc dữ liệu nội bộ (như đối tượng biểu diễn CV, Query, và kết quả xếp hạng). Các phân đoạn văn bản được giới hạn chặt chẽ về số lượng token nhằm tối ưu hóa cửa sổ ngữ cảnh của thuật toán. (Chi tiết đặc tả mã nguồn các cấu trúc dữ liệu này được trình bày tại Phụ lục A).

2.4.2. Tầng lưu trữ bền vững (Relational Database with JSONB)

Đối với việc lưu trữ kết quả phân tích và lịch sử xử lý, hệ thống sử dụng hệ quản trị cơ sở dữ liệu PostgreSQL.

- **Chiến lược phi chuẩn hóa (Denormalization):** Kết quả đầu ra của LLM mang tính chất bán cấu trúc (điểm số, danh sách kỹ năng, nhận xét tự do, câu hỏi phỏng vấn). Thay vì thiết kế nhiều bảng quan hệ rời rạc đòi hỏi các phép nối (JOIN) phức tạp, hệ thống đóng gói toàn bộ kết quả của một phiên phân tích thành một tài liệu JSON thống nhất và lưu trữ trong trường JSONB.
- **Tính linh hoạt:** Cách tiếp cận này giúp hệ thống dễ dàng mở rộng, thay đổi cấu trúc Prompt hoặc thêm bớt các tiêu chí đánh giá của AI trong tương lai mà không

cần thực hiện di chuyển dữ liệu (database migration). (Chi tiết lược đồ JSON đầu ra của LLM và cấu trúc bảng được đính kèm tại Phụ lục A).

2.5. Triển khai kiến trúc hệ thống

Trong các nghiên cứu ứng dụng AI, việc đánh giá mô hình trong môi trường tính thường chưa phản ánh đầy đủ khả năng vận hành trong thực tế. Do đó, hạ tầng phần mềm được triển khai trong đề tài này đóng vai trò như một môi trường thực thi (model serving infrastructure) và là minh chứng thực tiễn (proof-of-concept) cho hệ thống đề xuất. Hạ tầng này đảm bảo các thành phần thuật toán được trình bày trong Chương 3 có thể vận hành ổn định trong điều kiện thực tế, hỗ trợ xử lý dữ liệu theo thời gian thực và duy trì tính nhất quán của toàn bộ pipeline.

2.5.1. Kiến trúc xử lý bất đồng bộ (Asynchronous Architecture)

Hệ thống được xây dựng dựa trên kiến trúc hướng sự kiện (Event-driven) và xử lý bất đồng bộ (Async/Await) thông qua framework FastAPI (Ramírez, 2018). Thiết kế này đặc biệt quan trọng để giải quyết đặc thù của bài toán RAG: hệ thống không bị đóng băng tài nguyên khi phải chờ phản hồi từ các API của các LLM như GPT-4 (OpenAI, 2023) hay Claude 3 (Anthropic, 2024). Toàn bộ môi trường thực thi được đóng gói qua Docker (Merkel, 2014) nhằm đảm bảo tính tái lập (reproducibility) của thực nghiệm. (Danh sách công nghệ và thư viện chi tiết xem tại Phụ lục B).

2.5.2. Chiến lược tải trước mô hình (Model Pre-warming)

Việc tích hợp các mô hình học sâu vào hệ thống thời gian thực thường đối mặt với thách thức về độ trễ khởi tạo (cold start). Để khắc phục, hệ thống áp dụng cơ chế tải trước (Pre-warm). Thay vì khởi tạo mô hình Cross-Encoder tại thời điểm nhận truy vấn đầu tiên, hệ thống nạp mô hình vào bộ nhớ ngay tại vòng đời khởi động của máy chủ (lifespan startup). Điều này ngăn chặn rủi ro tranh chấp tài nguyên (race condition) khi hệ thống tiếp nhận một lô dữ liệu (batch) gồm nhiều hồ sơ đồng thời.

2.5.3. Cơ sở hạ tầng luồng dữ liệu thời gian thực (SSE)

Đối với nghiệp vụ phân tích hàng loạt (lên đến hàng chục hồ sơ cùng lúc), hệ thống tích hợp công nghệ truyền tải sự kiện trực tiếp Server-Sent Events (SSE). Bằng cách sử dụng hàng đợi bất đồng bộ (asyncio.Queue) để tách biệt tiến trình xử lý thuật toán và tiến trình trả kết quả, máy chủ có thể đẩy trực tiếp điểm số của từng ứng viên về giao diện ngay khi vừa tính toán xong. Điều này chứng minh hệ thống AI hoàn toàn đáp

ứng được yêu cầu về trải nghiệm người dùng trong thực tế nghiệp vụ nhân sự. (Đặc tả các REST API Endpoint xem tại Phụ lục C).

CHƯƠNG 3:

XÂY DỰNG MÔ HÌNH TRUY XUẤT ĐA TẦNG VÀ SUY LUẬN RÀNG BUỘC

3.1. Biểu diễn bài toán và kiến trúc pipeline dưới góc nhìn thuật toán

Như đã trình bày tại Chương 1, bài toán đối sánh CV-JD được mô hình hóa dưới dạng bài toán xếp hạng tài liệu có giám sát, với mục tiêu học một hàm xếp hạng:

$$f_{rank}: Q \times D \rightarrow \mathbb{R}$$

trong đó Q là không gian truy vấn (JD), D là không gian tài liệu (CV), và giá trị $f_{rank}(q, d)$ biểu diễn mức độ phù hợp giữa một cặp (q, d) , được sử dụng để xác định thứ tự ưu tiên giữa các ứng viên.

Tuy nhiên, trong thực tế, việc huấn luyện trực tiếp một hàm $f_{rank}(q, d)$ tối ưu trên toàn bộ không gian $Q \times D$ là không khả thi. Nguyên nhân xuất phát từ hai yếu tố chính. Thứ nhất, không gian dữ liệu có kích thước lớn và mang tính phi cấu trúc, khiến chi phí tính toán lớn của các mô hình học sâu trở nên khó kiểm soát, nếu áp dụng trực tiếp có thể vi phạm ràng buộc độ trễ [NFR-01] và làm suy giảm năng lực xử lý hàng loạt [FR-02]. Thứ hai, tính dị thể (heterogeneous) của các nguồn tín hiệu – bao gồm từ vựng (lexical signals), ngữ nghĩa (semantic signals), kỹ năng (skill-based signals) và cấu trúc hồ sơ (structural signals) – khiến việc tích hợp chúng trong một bước xử lý duy nhất trở nên kém hiệu quả. Do đó, thay vì tiếp cận bài toán dưới dạng một ánh xạ đơn, nghiên cứu này biểu diễn hàm xếp hạng dưới dạng hợp của hai phép biến đổi chính:

$$f(q, d) = f_{rerank} \circ f_{retrieval}(q, d)$$

trong đó:

- $f_{retrieval}$: ánh xạ sinh tập ứng viên (candidate generation), có nhiệm vụ xác định một tập con các tài liệu có khả năng liên quan cao đối với truy vấn q
- f_{rerank} : ánh xạ tinh chỉnh thứ hạng, thực hiện đánh giá chi tiết dựa trên các đặc trưng đa chiều

Bên cạnh hàm xếp hạng định lượng, hệ thống còn xây dựng một ánh xạ bổ sung:

$$g_{rag}(q, d) = f_{gen}(\text{Context}(q, d))$$

trong đó:

- $g_{rag}(q, d)$: hàm sinh các phân tích định tính (ví dụ: nhận xét, khoảng trống kỹ năng, câu hỏi phỏng vấn) dựa trên ngữ cảnh đã được xử lý từ các tầng trước,

không tham gia trực tiếp vào quá trình xếp hạng, mà đóng vai trò hỗ trợ nhằm tăng tính giải thích và hỗ trợ ra quyết định

- f_{gen} : hàm sinh ngôn ngữ có điều kiện (conditional text generation function), nhận đầu vào là ngữ cảnh có cấu trúc $\text{Context}(q, d)$ và sinh ra đầu ra dạng chuỗi ký tự thuộc không gian văn bản $\mathcal{T}$, về bản chất là một ánh xạ từ không gian biểu diễn $\mathcal{C}$ sang không gian chuỗi ký tự, không trực tiếp tối ưu một độ đo định lượng, mà thực hiện quá trình suy luận có điều kiện dựa trên thông tin đầu vào

Biểu diễn này cho phép phân rã bài toán ban đầu thành hai thành phần rõ ràng: một thành phần định lượng phục vụ xếp hạng và một thành phần định tính phục vụ suy luận, từ đó tránh sự pha trộn giữa hai loại mục tiêu khác nhau trong cùng một hàm.

Cách phân rã trên phản ánh chiến lược **coarse-to-fine optimization**, trong đó các bước xử lý ban đầu ưu tiên khả năng bao phủ (recall), trong khi các bước sau tập trung tối ưu độ chính xác (precision) và chất lượng suy luận. Cơ sở của chiến lược này xuất phát từ việc các mô hình có độ chính xác cao như Cross-Encoder hoặc LLM thường đi kèm với chi phí tính toán lớn. Do đó, việc áp dụng trực tiếp các mô hình này trên toàn bộ tập dữ liệu là không khả thi về mặt thực thi. Thay vào đó, các phương pháp truy xuất hiệu quả được sử dụng ở giai đoạn đầu nhằm thu hẹp không gian tìm kiếm xuống một tập ứng viên có độ bao phủ cao, tạo điều kiện cho các mô hình phức tạp hơn được áp dụng ở các bước sau với chi phí chấp nhận được.

Đáng chú ý, mỗi ánh xạ trong chuỗi biến đổi trên không chỉ tương ứng với một bước xử lý, mà còn gắn liền với một mục tiêu tối ưu cụ thể trong bài toán xếp hạng. Cụ thể $f_{retrieval}$ tập trung tối đa hóa khả năng bao phủ thông qua việc khai thác đồng thời lexical signals và semantic signals. Tiếp theo, f_{rerank} thực hiện tối ưu hóa thứ tự ưu tiên bằng cách tích hợp các đặc trưng đa nguồn vào một hàm chấm điểm thống nhất thông qua các mô hình Learning-to-Rank. Trong khi đó, ánh xạ $g_{rag}(q, d)$ đảm nhiệm việc sinh các phân tích có ý nghĩa, đồng thời đảm bảo tính trung thực thông qua việc ràng buộc ngữ cảnh đầu vào.

Việc tách biệt các mục tiêu tối ưu theo từng giai đoạn không chỉ giúp giảm độ phức tạp tính toán của hệ thống, mà còn tạo điều kiện để phân tích rõ vai trò của từng thành phần trong việc cải thiện chất lượng xếp hạng tổng thể.

3.2. Mô hình truy xuất kết hợp (Hybrid Retrieval Model)

Trong kiến trúc tổng thể, thành phần $f_{\text{retrieval}}(q, d)$ có nhiệm vụ ánh xạ truy vấn tuyển dụng q và tập tài liệu D về một tập ứng viên rút gọn $C \subset D$. Về bản chất, đây là giai đoạn ưu tiên tối đa hóa độ bao phủ (recall) trong chiến lược coarse-to-fine, nhằm đảm bảo các ứng viên tiềm năng không bị loại bỏ sớm trước khi chuyển sang các tầng xử lý có chi phí tính toán cao hơn.

Như đã đề cập ở các phần trước, trong miền dữ liệu tuyển dụng kỹ thuật (IT domain), các tín hiệu liên quan giữa CV và JD mang tính dị thể (heterogeneous). Cụ thể, hệ thống cần đồng thời nắm bắt sự trùng khớp từ vựng trực tiếp đối với các thuật ngữ công nghệ, cũng như sự tương đồng ngữ nghĩa trong các mô tả về kinh nghiệm và năng lực. Do đó, mô hình hybrid retrieval được lựa chọn nhằm song hành khai thác cả lexical signals và tín hiệu ngữ nghĩa semantic signals.

3.2.1. Chiến lược biểu diễn dữ liệu và phân đoạn hồ sơ

Thay vì xem mỗi hồ sơ ứng viên như một khối tài liệu liền mạch, hệ thống thực hiện phân rã mỗi tài liệu $d_i \in D$ thành tập các phân đoạn:

$$d_i = \{c_{\text{summary}}, c_{\text{experience}}, c_{\text{education}}\}$$

Việc phân đoạn theo các vùng thông tin (kỹ năng, kinh nghiệm, học vấn) dựa trên cấu trúc phổ biến của hồ sơ ứng viên trong thực tế tuyển dụng, thay vì thực hiện chia đều theo độ dài văn bản, nhằm bảo toàn tính toàn vẹn ngữ nghĩa của từng đơn vị thông tin.

Các ngưỡng cắt (chunk size) được thiết lập dựa trên cấu trúc ngữ nghĩa của CV:

- Đoạn kỹ năng và kinh nghiệm ($c_{\text{summary}}, c_{\text{experience}}$): giới hạn tối đa 512 ký tự
- Đoạn học vấn ($c_{\text{education}}$): giới hạn tối đa 256 ký tự

Việc lựa chọn các ngưỡng này nhằm đảm bảo mỗi phân đoạn đủ dài để giữ ngữ cảnh cần thiết, nhưng không quá dài gây loãng mật độ tín hiệu từ vựng. Đồng thời, việc chuẩn hóa độ dài giúp giảm thiểu sai lệch (length bias) trong các phép đo tương đồng ở các bước xử lý tiếp theo. Các giá trị này được xem như tham số thiết kế, nhằm cân bằng giữa khả năng biểu diễn và chi phí tính toán, thay vì là các ràng buộc cứng của hệ thống.

Thiết kế phân đoạn trong nghiên cứu này tập trung vào các vùng thông tin có mật độ tín hiệu cao nhất đối với bài toán tuyển dụng, trong khi các thành phần khác như dự án (projects) hoặc giải thưởng (awards) thường được biểu diễn lồng trong các phân đoạn chính và do đó vẫn được khai thác gián tiếp.

3.2.2. Khai thác tín hiệu từ vựng (Lexical Retrieval)

Trong miền dữ liệu tuyển dụng kỹ thuật, các thuật ngữ đặc thù (ví dụ: Golang, ReactJS, Kubernetes, ...) thường xuất hiện dưới dạng gần như cố định giữa JD và CV. Do đó, khả năng so khớp chính xác ở mức từ vựng đóng vai trò quan trọng trong việc nhận diện các kỹ năng cốt lõi.

Hệ thống sử dụng thuật toán Okapi BM25 để khai thác các tín hiệu này trên không gian các phân đoạn:

$$score_{BM25}(q, c_j) = \sum_{t \in q} IDF(t) \cdot \frac{f(t, c_j) \cdot (k_1 + 1)}{f(t, c_j) + k_1 \cdot \left(1 - b + b \cdot \frac{|c_j|}{avgdl}\right)}$$

Các hyperparameter của mô hình được thiết lập trong khoảng giá trị thường được sử dụng trong các hệ thống truy xuất thông tin, nhằm đảm bảo sự cân bằng giữa độ nhạy của tần suất từ và cơ chế chuẩn hóa độ dài tài liệu. Cụ thể:

- $k_1 = 1.5$: kiểm soát mức độ bão hòa của tần suất từ, tránh việc một từ khóa lặp lại quá nhiều lần gây thiên lệch điểm số
- $b = 0.75$: duy trì cơ chế chuẩn hóa độ dài tài liệu, giúp điều chỉnh ảnh hưởng của các phân đoạn có độ dài bất thường

Mặc dù trong phạm vi nghiên cứu này, dữ liệu đã được phân đoạn với độ dài được kiểm soát, các phân đoạn vẫn không đảm bảo hoàn toàn đồng nhất về độ dài và mật độ thông tin. Các đoạn mô tả kinh nghiệm thường có xu hướng dài hơn và chứa nhiều thông tin không liên quan trực tiếp đến yêu cầu công việc, trong khi các đoạn kỹ năng hoặc học vấn có xu hướng ngắn gọn và tập trung hơn. Do đó, $b = 0.75$ vẫn đóng vai trò quan trọng trong việc điều chỉnh ảnh hưởng của độ dài, giúp hạn chế việc các phân đoạn dài nhưng chứa nhiều thông tin dư thừa chi phối điểm số truy xuất. Thiết kế này đảm bảo rằng điểm BM25 phản ánh mức độ liên quan thực chất, thay vì bị ảnh hưởng bởi độ dài tuyệt đối của văn bản.

3.2.3. Khai thác tín hiệu ngữ nghĩa (Dense Retrieval)

Để xử lý hiện tượng bất đồng từ vựng, hệ thống bổ sung một luồng truy xuất ngữ nghĩa dựa trên biểu diễn embedding.

Truy vấn q và phân đoạn c_j được ánh xạ vào cùng một không gian vector $\mathbb{R}^d$, trong đó độ tương đồng được đo bằng cosine similarity:

$$\text{score}_{dense}(q, c_j) = \frac{v_q \cdot v_{c_j}}{\|v_q\| \cdot \|v_{c_j}\|}$$

Mô hình embedding được lựa chọn theo hướng cân bằng giữa khả năng biểu diễn ngữ nghĩa và tính khả thi trong triển khai thực nghiệm. Cụ thể, hệ thống sử dụng mô hình **BGE-large-en-v1.5** (Xiao và cộng sự, 2023) với không gian embedding kích thước $d = 1024$, cho phép biểu diễn tốt các khái niệm kỹ thuật phức tạp trong bài toán tuyển dụng.

Ngoài ra, thiết kế embedding được xây dựng theo hướng mã hóa bất đối xứng giữa truy vấn và tài liệu, phù hợp với sự khác biệt về cấu trúc ngôn ngữ giữa JD (ngắn, mang tính yêu cầu) và CV (dài, mang tính mô tả). Điều này giúp cải thiện khả năng đo lường tương đồng ngữ nghĩa trong bối cảnh hai loại văn bản có phân phối khác nhau.

3.2.4. Dung hợp thứ hạng bằng RRF

Hai luồng truy xuất tạo ra các điểm số có bản chất khác nhau, dẫn đến khó khăn trong việc chuẩn hóa trực tiếp. Do đó, hệ thống áp dụng phương pháp dung hợp dựa trên thứ hạng thông qua RRF:

$$\text{score}_{RRF} = \frac{1}{k + \text{rank}_{BM25}(c_j)} + \frac{1}{k + \text{rank}_{dense}(c_j)}$$

trong đó $\text{rank}_{BM25}(c_j)$ và $\text{rank}_{dense}(c_j)$ lần lượt là thứ hạng của phân đoạn c_j trong hai danh sách truy xuất tương ứng.

Hyperparameter k được thiết lập trong khoảng giá trị thường được sử dụng trong các hệ thống truy xuất thông tin. Trong nghiên cứu này, $k = 60$ được lựa chọn nhằm đảm bảo sự ổn định trong quá trình dung hợp thứ hạng giữa các nguồn truy xuất có bản chất khác nhau. Giá trị này giúp làm giảm độ nhạy của hàm hợp nhất đối với các thay đổi nhỏ ở thứ hạng thấp, đồng thời ưu tiên các phân đoạn có thứ hạng cao một cách nhất quán trên nhiều phương pháp.

Quá trình dung hợp được thực hiện ở cấp độ phân đoạn nhằm tận dụng tối đa các tín hiệu cục bộ trước khi ánh xạ về cấp độ tài liệu.

3.2.5. Ánh xạ từ phân đoạn sang hồ sơ

Do truy xuất được thực hiện ở cấp độ phân đoạn, hệ thống cần ánh xạ kết quả về cấp độ hồ sơ thông qua toán tử cực đại:

$$\text{score}_{retrieval}(q, d_i) = \max_{c_j \in d_i} \text{score}_{RRF}(c_j)$$

Lựa chọn toán tử cực đại phản ánh giả định rằng một ứng viên được xem là phù hợp nếu tồn tại ít nhất một thành phần trong hồ sơ thể hiện sự tương thích mạnh với yêu

cầu công việc. Cách tiếp cận này đặc biệt phù hợp trong bối cảnh thông tin liên quan thường tập trung cục bộ trong một số phần nhất định của CV (ví dụ: kinh nghiệm hoặc kỹ năng), thay vì phân bố đồng đều trên toàn bộ tài liệu.

Mặc dù trong một số trường hợp hiếm, tín hiệu mạnh có thể xuất phát từ các phân đoạn ít mang tính quyết định (chẳng hạn như học vấn), việc kết hợp với các tầng xử lý phía sau (Learning-to-Rank và Re-ranking) giúp tái đánh giá và hiệu chỉnh thứ hạng dựa trên các đặc trưng toàn cục của hồ sơ. Do đó, phép cực đại được sử dụng như một bước phát hiện tín hiệu tiềm năng ở giai đoạn truy xuất, thay vì quyết định cuối cùng về mức độ phù hợp.

3.2.6. Vai trò trong pipeline tổng thể

Tầng truy xuất không nhằm tối ưu trực tiếp thứ hạng cuối cùng, mà đóng vai trò tạo ra một tập ứng viên có độ bao phủ cao. Việc kết hợp giữa phân đoạn ngữ nghĩa, truy xuất từ vựng, truy xuất ngữ nghĩa và dung hợp thứ hạng cho phép hệ thống khai thác hiệu quả các nguồn tín hiệu dị thể trong dữ liệu CV-JD.

Tập ứng viên sinh ra từ giai đoạn này đóng vai trò đầu vào cho tầng học xếp hạng f_{rerank} , nơi các mô hình có độ phức tạp cao hơn được áp dụng nhằm tối ưu hóa thứ tự ưu tiên.

3.3. Mô hình Learning-to-Rank và Re-ranking

Sau giai đoạn hybrid retrieval, hệ thống thu được tập ứng viên rút gọn $C \subset D$ có kích thước $N = |C|$ đã được xếp hạng sơ bộ dựa trên điểm số $score_{retrieval}$. Tuy nhiên, các điểm số tại giai đoạn này chỉ phản ánh mức độ tương đồng văn bản, chưa đủ để biểu diễn mức độ phù hợp nghiệp vụ trong bối cảnh tuyển dụng. Do đó, hệ thống xây dựng một hàm tái xếp hạng tổng quát:

$$f_{rerank}(q, d_i) = score_{final}(q, d_i)$$

trong đó $f_{rerank}(q, d_i)$ biểu diễn điểm số cuối cùng của ứng viên d_i đối với truy vấn q , được sử dụng để xác định thứ tự ưu tiên trong danh sách kết quả. Trong nghiên cứu này, thay vì được xây dựng như một mô hình đơn lẻ, hàm f_{rerank} được triển khai theo kiến trúc **nối tiếp có điều chỉnh (adaptive cascade)**, kết hợp giữa xử lý tuần tự và tổ hợp điểm số:

$$f_{rerank} = f_{fusion} \left(f_{CE}(q, d_i), f_{LTR}(\phi(q, d_i)) \right)$$

Cụ thể, quá trình xử lý diễn ra theo chiến lược **coarse-to-fine optimization**:

- Hàm Learning-to-Rank f_{LTR} được áp dụng trên toàn bộ tập $\mathcal{C}$ để đánh giá đa chiều và xếp hạng sơ bộ
- Hệ thống thực hiện thu hẹp không gian tìm kiếm, giữ lại tập con $\mathcal{C}' = \text{Top-}K(\mathcal{C})$, với $K < N$.
- Hàm Re-ranking f_{CE} được kích hoạt trên tập $\mathcal{C}'$ để tinh chỉnh thứ hạng ở mức độ ngữ nghĩa phân giải cao.
- Cuối cùng, hai nguồn tín hiệu được tổ hợp qua hàm f_{fusion} để sinh ra điểm số quyết định.

Trong thiết kế này, kích thước không gian đầu vào N và ngưỡng cắt chuyển tiếp K được mô hình hóa như các hyperparameter cấu hình động. Việc không ấn định cứng các giá trị này đảm bảo tính tổng quát của thuật toán, cho phép hệ thống linh hoạt mở rộng quy mô (tăng K để tối ưu độ chính xác) hoặc thu hẹp (giảm K để tối ưu độ trễ) tùy thuộc vào giới hạn tài nguyên tính toán khi triển khai thực tế.

3.3.1. Thiết kế không gian đặc trưng (Feature Engineering)

Thành phần đầu tiên trong chuỗi nối tiếp là việc ánh xạ cặp (q, d_i) thành một feature vector định lượng. Nghiên cứu đề xuất không gian đặc trưng $\mathcal{F} \subset \mathbb{R}^{10}$, với vector $\phi(q, d_i) = [x_1, x_2, \dots, x_{10}]^T$ được thiết kế dựa trên bản chất ngữ nghĩa của từng loại thông tin:

- **Nhóm A: Tín hiệu truy xuất nền tảng (Base retrieval signals)**

Kế thừa từ tầng truy xuất, được chuẩn hóa để đảm bảo tính ổn định cho không gian gradient:

- **x_1 (BM25 Score):** Điểm BM25 là một giá trị không bị chặn trên $[0, \infty)$. Nếu dùng chuẩn hóa Min-Max tuyến tính, mô hình sẽ bị bóp méo nghiêm trọng bởi các hồ sơ ngoại lai (cố tình nhồi nhét từ khóa để đạt điểm cao). Do đó, hệ thống áp dụng soft-normalization $x_1 = \frac{score_{BM25}}{score_{BM25} + c}$. Trong nghiên cứu này, hệ số $c = 5$ được chọn đóng vai trò là một hệ số tỷ lệ (scaling factor). Trong thực tế, điểm BM25 của các phân đoạn có liên quan thường phân bố trong một khoảng hữu hạn và có xu hướng hội tụ ở vùng trung bình, trong khi các giá trị ngoại lai có thể tăng đột biến do hiện tượng nhồi nhét từ khóa. Việc chọn $c = 5$ giúp ánh xạ các giá trị này vào vùng biến thiên nhạy cảm nhất của hàm phân thức (khoảng 0.5 – 0.75), đồng

thời nén chặt các giá trị ngoại lai tiệm cận về 1.0 mà không làm mất đi phương sai của dữ liệu.

- **x_2 (Dense Score):** Điểm cosine tự nhiên nằm trong khoảng $[-1,1]$. Tuy nhiên, đối với các mô hình embedding hiện đại như BGE-large, phân phối điểm số thường bị dồn cụm (clustering) ở mức cao (thường >0.4 đối với ngay cả các văn bản ít liên quan). Hệ thống sử dụng hàm Sigmoid có điều chỉnh nhiệt độ (temperature scaling) $x_2 = \frac{1}{e^{-score_{dense}/\tau}}$. Trong nghiên cứu này, hyperparameter $\tau = 3.0$ được cấu hình ở mức trung bình nhằm làm mượt độ dốc của hàm Sigmoid truyền thống, tránh việc điểm số bị bão hòa quá nhanh về 0 hoặc 1, qua đó giữ lại được sự khác biệt tinh tế giữa các vector ngữ nghĩa.

- **Nhóm B: Tín hiệu tương thích kỹ năng (Skill-based signals)**

- **x_3 (Hard Skill Jaccard):** Kỹ năng cứng mang tính định danh tuyệt đối (ví dụ: Java, PostgreSQL). Việc dùng khoảng cách vector có thể gây nội suy sai do các từ gần nghĩa (ví dụ: Java và JavaScript). Hệ thống dùng chỉ số Jaccard trên tập hợp để đo độ giao thoa chính xác $x_3 = \frac{|S_{q_{hard}} \cap S_{d_{hard}}|}{|S_{q_{hard}} \cup S_{d_{hard}}|}$.
- **x_4 (Soft Skill Cosine):** Kỹ năng mềm có tính đa hình cao trong diễn đạt. Cosine similarity của vector TF-IDF được sử dụng nhằm cho phép dung sai ngữ nghĩa.
- **x_5 (Skill Coverage):** Phạt các hồ sơ ứng viên thiếu hụt yêu cầu bắt buộc $x_5 = \frac{|S_{required} \cap S_d|}{|S_{required}|}$.

- **Nhóm C: Tín hiệu cấu trúc và thâm niên (Structural signals)**

- **x_6 (Seniority Match):** Khoảng cách cấp bậc giữa yêu cầu và thực tế. Các cấp bậc được mã hóa thành số nguyên (từ Intern=0 đến Manager=5). Phép mã hóa này mang tính xấp xỉ tuyến tính nhằm đơn giản hóa biểu diễn và cho phép mô hình Learning-to-Rank học lại trọng số phù hợp từ dữ liệu.
- **x_7 (Experience Ratio):** Để tránh hiện tượng mô hình bị thiên lệch bởi các hồ sơ ứng viên quá dư thừa kinh nghiệm (overqualified bias), hệ thống áp dụng hàm chặn trên $x_7 = \min\left(\frac{exp_{cv}}{exp_{jd_{min}}}, 1\right)$.

- **x_8 (Education Match):** Biến nhị phân (0 hoặc 1) phản ánh việc đáp ứng trình độ học vấn.
- **Nhóm D: Tín hiệu từ vựng bổ trợ (Auxiliary lexical signals)**
 - **x_9 (Title BM25):** Điểm so khớp chức danh yêu cầu và chức danh thực tế trong lịch sử làm việc.
 - **x_{10} (Skills TF-IDF Cosine):** Bổ trợ cho x_3 , phép toán này gán trọng số IDF cao cho các kỹ năng hiếm gặp. Phép biến đổi này dựa trên nguyên lý của lý thuyết thông tin, trong đó các thuật ngữ hiếm mang lượng thông tin cao hơn, từ đó giúp tăng khả năng phân biệt giữa các ứng viên.

3.3.2. Xếp hạng sơ bộ với LambdaMART (First-stage Re-ranking)

Với đầu vào là vector đặc trưng $\phi(q, d_i)$, hệ thống sử dụng thuật toán LambdaMART để học hàm ánh xạ f_{LTR} . Khác với các mô hình phân loại hay hồi quy thông thường, LambdaMART giải quyết bài toán tối ưu hóa thứ tự danh sách (listwise ranking).

Do các độ đo xếp hạng như NDCG không khả vi (non-differentiable), LambdaMART không tối ưu trực tiếp hàm mục tiêu này, mà xấp xỉ gradient thông qua các lambda gradient λ_{ij} . Các giá trị này đóng vai trò như pseudo-gradients, được xây dựng từ đạo hàm của hàm mất mát pairwise (logistic loss), có trọng số theo độ thay đổi của NDCG khi hoán đổi thứ hạng của một cặp ứng viên (d_i, d_j) trong đó d_i có mức độ liên quan cao hơn d_j

$$\lambda_{ij} = \frac{-\sigma}{1 + e^{\sigma(s_i - s_j)}} |\Delta \text{NDCG}|$$

trong đó s_i, s_j là điểm số do mô hình dự đoán, σ là hyperparameter điều chỉnh độ dốc của hàm logistic, kiểm soát mức độ nhạy của gradient đối với sai khác điểm số, và $|\Delta \text{NDCG}|$ biểu diễn mức thay đổi tuyệt đối của NDCG khi hoán đổi vị trí của hai ứng viên. Cơ chế này ép mô hình phải tập trung tối ưu hóa và đẩy các hồ sơ xuất sắc lên các vị trí đầu tiên của danh sách.

Trong kiến trúc cascade của hệ thống, hàm f_{LTR} đóng vai trò là tầng lọc định lượng tính toán điểm số $score_{LTR}(q, d_i)$ cho toàn bộ N hồ sơ trong tập C . Dựa trên điểm số này, hệ thống thực hiện thao tác cắt tỉa (prunning), chỉ giữ lại tập K ứng viên xuất sắc nhất để tạo thành tập $C' = \text{Top-}K(C)$ chuyển giao sang giai đoạn tái xếp hạng sâu.

3.3.3. Tái xếp hạng ngữ nghĩa sâu với Cross-Encoder (Deep semantic Re-ranking)

Mặc dù LambdaMART thể hiện hiệu năng cao trong việc tối ưu hóa tổ hợp các đặc trưng tính $\phi(q, d_i)$, mô hình này vẫn tồn tại một hạn chế cơ bản: thiếu khả năng mô hình hóa tương tác trực tiếp ở cấp độ ngữ cảnh sâu (token-level interaction) giữa JD và CV. Để khắc phục hạn chế này, hệ thống tích hợp kiến trúc Cross-Encoder như một lớp re-ranking cuối cùng. Trong nghiên cứu này, mô hình Cross-Encoder được xây dựng dựa trên kiến trúc Transformer, sử dụng biến thể ms-marco-MiniLM-L-6-v2 (Wang và cộng sự, 2020) đã được huấn luyện cho các tác vụ xếp hạng và hỏi đáp trên tập dữ liệu quy mô lớn MS MARCO (Bajaj và cộng sự, 2018). Khác với kiến trúc Bi-Encoder ở tầng truy xuất – nơi truy vấn q và tài liệu d_i được mã hóa thành hai vector độc lập – Cross-Encoder nhận đầu vào dưới dạng một chuỗi kết hợp:

$$\text{Input} = [\text{CLS}] \, q \, [\text{SEP}] \, d_i \, [\text{SEP}]$$

Thiết kế này cho phép cơ chế Self-Attention tính toán sự phụ thuộc giữa mọi cặp token thuộc JD và CV ngay từ các hidden layer đầu tiên. Nhờ đó, mô hình có khả năng nắm bắt các quan hệ ngữ nghĩa ở mức độ chi tiết, ví dụ như việc một kỹ năng được sử dụng trong ngữ cảnh cụ thể của một dự án, thay vì chỉ dựa trên sự xuất hiện của từ khóa.

Đầu ra của mô hình là một giá trị vô hướng $\text{score}_{CE}(q, d_i)$ biểu diễn mức độ liên quan giữa truy vấn và tài liệu.

Về mặt tính toán, cơ chế Self-Attention có độ phức tạp bậc hai theo chiều dài chuỗi đầu vào: $\mathcal{O}(|q| + |d_i|)^2$. Do đó, việc áp dụng Cross-Encoder trên toàn bộ không gian tài liệu D hoặc thậm chí trên tập ứng viên C là chưa khả thi, có thể vi phạm trực tiếp ràng buộc [NFR-01] và làm suy giảm nghiêm trọng khả năng xử lý hàng loạt [FR-02]. Điều này dẫn đến việc lựa chọn kiến trúc cascade trong thiết kế hệ thống. Cụ thể, LambdaMART đóng vai trò như một tầng lọc định lượng chi phí thấp, thu hẹp không gian tìm kiếm, sau đó, Cross-Encoder chỉ được áp dụng trên tập này để thực hiện tái xếp hạng chi tiết. Thiết kế này cho phép hệ thống đạt được sự cân bằng giữa độ chính xác của các mô hình học sâu và chi phí tính toán, đồng thời đảm bảo khả năng vận hành trong các điều kiện tài nguyên thực tế.

3.3.4. Tổ hợp điểm số và Phân ngưỡng quyết định (Final score fusion)

Đối với các ứng viên xuất sắc nhất lọt vào tập C' (Top- K), hệ thống đã thu thập được hai hệ thống tín hiệu định lượng mang bản chất khác biệt: score_{LTR} phản ánh các quy luật học được từ dữ liệu thông qua mô hình Learning-to-Rank, trong khi score_{CE}

đại diện cho khả năng mô hình hóa tương tác ngữ nghĩa sâu và tương tác từ vựng sâu từ mô hình học sâu. Để đưa ra quyết định xếp hạng chung cuộc, hệ thống định nghĩa điểm số cuối cùng thông qua một hàm tổ hợp tuyến tính:

$$score_{final}(q, d_i) = \alpha \cdot score_{LTR}(q, d_i) + (1 - \alpha) \cdot score_{CE}(q, d_i)$$

trong đó $\alpha \in [0,1]$ là hyperparameter nội suy đóng vai trò kiểm soát trọng số tín nhiệm giữa hai luồng mô hình.

- Khi $\alpha \rightarrow 1$, hệ thống ưu tiên ứng viên dựa trên các khối kỹ năng cứng và cấu trúc số năm kinh nghiệm rõ ràng.
- Khi $\alpha \rightarrow 0$, hệ thống trao quyền quyết định cho khả năng phân tích ngữ nghĩa sâu, ưu tiên các ứng viên có cách hành văn khớp với yêu cầu công việc.

Việc duy trì song song hai luồng điểm số thông qua α là một thiết kế mang tính ổn định (robustness-oriented design). Cách tiếp cận này giúp hệ thống hạn chế ảnh hưởng của các tín hiệu bề mặt (surface-level signals) không phản ánh đúng năng lực thực tế, đồng thời vẫn giữ được khả năng phát hiện các ứng viên tiềm năng thông qua phân tích ngữ nghĩa sâu.

Cuối cùng, $score_{final}$ được chuẩn hóa và sử dụng làm đầu vào cho một hàm phân ngưỡng (thresholding function), nhằm ánh xạ điểm số liên tục thành các quyết định rời rạc, bao gồm: Shortlist (đủ điều kiện), Hold (xem xét thêm) và Reject (từ chối). Kết quả phân loại này đóng vai trò là đầu vào cho tầng sinh ngôn ngữ, nơi các quyết định định lượng được diễn giải thành các phân tích định tính phục vụ quá trình ra quyết định.

3.4. Cơ chế suy luận và sinh ngôn ngữ có ràng buộc (Faithful RAG)

Sau khi hoàn tất giai đoạn Re-ranking, hệ thống xác định được tập ứng viên C' cùng với điểm số định lượng $score_{final}(q, d_i)$. Tuy nhiên, trong bối cảnh tuyển dụng, các giá trị định lượng này chưa đủ để đưa ra quyết định, vốn đòi hỏi các phân tích định tính như nhận xét tổng hợp, xác định các khoảng trống kỹ năng và đề xuất câu hỏi phỏng vấn.

Để đáp ứng yêu cầu này, hệ thống tích hợp LLM nhằm thực hiện suy luận và sinh ngôn ngữ. Tuy nhiên, các mô hình LLM tự hồi quy (autoregressive) tiềm ẩn rủi ro sinh ra thông tin không có căn cứ (hallucination), đặc biệt nghiêm trọng trong các ứng dụng liên quan đến hồ sơ cá nhân. Để khắc phục lỗ hổng này và đáp ứng được quy định tại [NFR-04], nghiên cứu đề xuất kiến trúc RAG có ràng buộc (Faithful RAG) nhằm đảm bảo tính trung thực của đầu ra.

3.4.1. Thiết kế bối cảnh có cấu trúc (Structured context grounding)

Trong RAG truyền thống, bối cảnh (context) đưa vào LLM thường là văn bản thô (raw text). Tuy nhiên, CV thô chứa rất nhiều nhiễu định dạng, làm tăng rủi ro mô hình nhận diện sai thực thể.

Vì vậy, hệ thống đề xuất chiến lược bối cảnh có cấu trúc. Thay vì cung cấp toàn văn CV, hệ thống chỉ đẩy vào LLM một tập dữ liệu đã được cấu trúc hóa dưới định dạng JSON (bao gồm danh sách kỹ năng, số năm kinh nghiệm, học vấn đã trích xuất từ tầng trước). Về mặt lý thuyết thông tin (Information Theory), JSON đóng vai trò như một cơ chế lọc nhiễu nghiêm ngặt. Việc sử dụng các cặp Key-Value rõ ràng giúp thu hẹp không gian chú ý (attention space) của mạng Transformer, ép mô hình phải neo (ground) sự chú ý vào các sự kiện cụ thể thay vì bị phân tâm bởi các từ ngữ hoa mỹ.

3.4.2. Tối ưu hóa generation hyperparameter

Quá trình sinh từ vựng của LLM là việc lấy mẫu từ một phân phối xác suất, được kiểm soát bởi temperature hyperparameter τ thông qua hàm Softmax (Hinton và cộng sự, 2015):

$$P(y_i|x) = \frac{\exp\left(\frac{z_i}{\tau}\right)}{\sum_j \exp\left(\frac{z_j}{\tau}\right)}$$

trong đó z_i là logit của token i . Khi $\tau \rightarrow 1$, phân phối trở nên phẳng hơn, làm tăng tính đa dạng nhưng đồng thời gia tăng rủi ro sinh thông tin không chính xác. Ngược lại, khi $\tau \rightarrow 0$, phân phối tập trung hơn, làm tăng tính quyết định của đầu ra.

Trong bối cảnh bài toán tuyển dụng, việc sinh thông tin sai lệch là không chấp nhận được. Do đó trong nghiên cứu này, hệ thống cấu hình $\tau = 0.2$ ở mức thấp. Thiết kế này giúp làm giảm đáng kể mức độ đa dạng của phân phối đầu ra, tăng mức độ bám sát ngữ cảnh đầu vào.

3.4.3. Cơ chế Prompt ràng buộc

Bên cạnh việc kiểm soát dữ liệu đầu vào và tham số sinh, hệ thống áp dụng cơ chế **ràng buộc hành vi mô hình thông qua prompt**. Để điều khiển LLM với độ chính xác cao nhất, hệ thống sử dụng tiếng Anh làm ngôn ngữ thiết kế prompt. Do các mô hình LLM hiện đại được huấn luyện chủ yếu trên ngữ liệu tiếng Anh, việc sử dụng tiếng Anh trong thiết kế prompt giúp cải thiện tính ổn định trong tuân thủ mệnh lệnh và suy luận logic.

Trong nghiên cứu này, hệ thống triển khai cơ chế prompt dựa trên ràng buộc phủ định (negative constraints) để hạn chế tối đa cơ chế nội suy của mô hình:

- **System Persona:** "**You are an expert IT Recruiter evaluating candidates for tech roles.**" (Phân vai hệ thống: “Bạn là một chuyên gia tuyển dụng IT, thực hiện đánh giá các ứng viên cho các vị trí công việc trong lĩnh vực công nghệ.”).
- **Negative Constraints:** "**STRICTLY base your evaluation ONLY on the provided JSON context. DO NOT hallucinate, infer, or assume any skills or experiences not explicitly listed. If an information is missing, clearly state 'Not provided'.**" (Ràng buộc phủ định: “NGHIÊM TÚC căn cứ kết quả đánh giá CHỈ dựa trên nội dung JSON được cung cấp. KHÔNG được ảo giác (hallucinate), suy diễn hoặc tự ý giả định bất kỳ kỹ năng hay kinh nghiệm nào không được liệt kê rõ ràng. Nếu thông tin bị thiếu, hãy ghi rõ 'Không được cung cấp'.”).
- **Formatting:** "**Return your analysis in valid JSON format. Provide the final reasoning in Vietnamese.**" (Định dạng đầu ra: “Trả về kết quả phân tích dưới định dạng JSON hợp lệ. Phần lập luận cuối cùng phải được trình bày bằng tiếng Việt.”).

3.4.4. Đánh giá sơ bộ làm cơ sở thiết kế (Preliminary validation)

Để bảo vệ cho các quyết định thiết kế nêu trên, một thử nghiệm sơ bộ trong giai đoạn xây dựng hệ thống đã được thực hiện nhằm đánh giá mức độ trung thực của tầng sinh ngôn ngữ.

Kết quả cho thấy sự kết hợp của ba yếu tố: bối cảnh có cấu trúc dưới dạng JSON, kiểm soát nhiệt độ sinh ở mức thấp và cơ chế prompt ràng buộc giúp hệ thống đạt độ đo Faithfulness xấp xỉ 0.977, tiệm cận mức tối đa. Kết quả này được sử dụng như một bằng chứng thực nghiệm sơ bộ (preliminary validation) nhằm hỗ trợ việc lựa chọn kiến trúc Faithful RAG.

CHƯƠNG 4: THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. Thiết lập thực nghiệm và ràng buộc tài nguyên

4.1.1. Môi trường và cấu hình thực nghiệm

Nhằm kiểm chứng các giả thuyết thuật toán đã được đề xuất trong Chương 3, toàn bộ pipeline thực nghiệm được triển khai trong môi trường cô lập thông qua Docker (Python 3.11-slim), đảm bảo tính tái lập (reproducibility) và tính nhất quán (consistency) giữa các lần đánh giá. Các thành phần trong pipeline tương ứng trực tiếp với các ánh xạ đã định nghĩa:

- $f_{retrieval}$: BM25 ($k_1 = 1.5, b = 0.75$) và Dense Retrieval (BGE-large-en-v1.5)
- f_{rerank} : LambdaMART và Cross-Encoder (ms-marco-MiniLM-L-6-v2)
- g_{rag} : GPT-4o-mini với temperature $\tau = 0.2$, đầu ra JSON
- Để đảm bảo ràng buộc về latency thực nghiệm, không gian cắt tĩa được thiết lập ở mức $K = 50$. Trọng số nội suy của hàm f_{fusion} là $\alpha = 0.3$ ưu tiên khả năng nắm bắt ngữ nghĩa sâu.

Thiết lập này đảm bảo tính nhất quán giữa mô hình lý thuyết đã trình bày ở Chương 3 và môi trường thực nghiệm.

4.1.2. Ràng buộc tài nguyên và phạm vi thực nghiệm

Trong khuôn khổ khóa luận, hệ thống được triển khai dưới một số ràng buộc thực tế: Không sử dụng hạ tầng GPU chuyên dụng và giới hạn số lượng CV xử lý trong mỗi phiên phân tích hàng loạt (batch) tối đa 20 hồ sơ. Giới hạn 20 CV chỉ áp dụng ở mức độ xử lý thời gian thực (runtime) nhằm đảm bảo hiệu năng và giới hạn API, hoàn toàn không phản ánh quy mô tập dữ liệu dùng cho thực nghiệm. Các thí nghiệm định lượng về hiệu năng IR được thực hiện trên tập dữ liệu lớn hơn.

Do giới hạn về quy mô dữ liệu huấn luyện, thay vì theo đuổi một điểm số tối đa tuyệt đối (vốn đòi hỏi tài nguyên đồ sộ), các thực nghiệm trong nghiên cứu này được thiết kế theo phương pháp Phân tích bóc tách (Ablation Study). Mục tiêu cốt lõi là đánh giá sự đóng góp độc lập của từng thành phần kiến trúc (BM25, Dense, Cross-Encoder, LambdaMART) vào hiệu năng tổng thể. Cách tiếp cận này giúp kiểm chứng bằng thực nghiệm những luận điểm đã thiết kế ở Chương 3, xác nhận tính đúng đắn của việc phân rã bài toán thành chuỗi ánh xạ $f(q, d) = f_{rerank} \circ f_{retrieval}(q, d)$ đã thiết kế.

4.2. Bộ dữ liệu và chiến lược Ground Truth

4.2.1. Tập dữ liệu thực nghiệm và Ground Truth

Thực nghiệm sử dụng hai nguồn dữ liệu chính để đánh giá độc lập hai nhánh của pipeline:

- **Nhánh xếp hạng ($f_{retrieval}, f_{rerank}$):** Sử dụng **2.000 hồ sơ** trích xuất từ tập dữ liệu **ResumeAtlas** (Heakl và cộng sự, 2024). Trong bối cảnh thiếu hụt nhãn đánh giá thủ công từ chuyên gia nhân sự (Human Annotation) trên quy mô lớn, nghiên cứu áp dụng kỹ thuật **Gán nhãn giả định dựa trên tri thức chuyên ngành** (Domain-heuristic Pseudo-relevance Labeling). Cụ thể, Ground Truth được xây dựng thông qua việc kết hợp đối sánh danh mục (Category matching) và độ phủ từ khóa kỹ năng (Keyword overlap) để nội suy ra 3 mức độ liên quan: Shortlist (2), Hold (1), Reject (0). Kỹ thuật giám sát yếu (Weak Supervision) này là phương pháp tiếp cận tiêu chuẩn trong nghiên cứu Truy xuất thông tin (IR), giúp tạo ra một bộ Ground Truth tiệm cận (proxy) đủ lớn để đánh giá tính ổn định của các thuật toán xếp hạng.
- **Nhánh sinh ngôn ngữ (g_{rag}):** Sử dụng **100 mẫu ngẫu nhiên** từ tập kiểm thử **SQuAD v1.1** (Rajpurkar và cộng sự, 2016). Các cặp (đoạn văn, câu hỏi) được ánh xạ vào vai trò (Ngữ cảnh CV+JD, Câu hỏi phỏng vấn) nhằm đánh giá độ trung thực của tầng sinh ngôn ngữ (Faithfulness) dựa trên văn bản có sẵn, giảm thiểu sự phụ thuộc vào dữ liệu gán nhãn nghiệp vụ đắt đỏ.

4.2.2. Chiến lược phân chia dữ liệu (Train/Test Split)

Để đảm bảo tính khách quan và ngăn chặn rủi ro quá khớp (overfitting), đặc biệt là đối với mô hình có tính chất học máy như LambdaMART, tập dữ liệu ResumeAtlas (2.000 hồ sơ) được phân chia nghiêm ngặt theo chiến lược 80-20.

- **Tập huấn luyện (Training Set - 80%):** Bao gồm 1600 hồ sơ và các truy vấn tương ứng, được sử dụng duy nhất để huấn luyện các cây quyết định trong mô hình LambdaMART, giúp mô hình học cách thiết lập trọng số cho các đặc trưng (features) đã thiết kế ở Chương 3.
- **Tập kiểm thử (Testing Set - 20%):** Bao gồm 400 hồ sơ, được cách ly hoàn toàn trong quá trình huấn luyện. Tập này được sử dụng để chạy thực nghiệm định lượng và trích xuất các điểm số NDCG, MAP, MRR nhằm đánh giá hiệu năng thực tế của hệ thống đối với dữ liệu chưa từng gặp (unseen data).

4.2.3. Hệ thống độ đo đánh giá

Cách phân tách hệ thống độ đo phản ánh trực tiếp cấu trúc bài toán:

- **Nhóm IR Metrics:** NDCG@10, MAP@10, MRR được tính toán qua thư viện `ranx` (Bassani, 2022) để đánh giá nhánh xếp hạng.
- **Nhóm RAGAS:** Faithfulness, Context Precision, Context Recall, Answer Relevancy được sử dụng để đánh giá nhánh sinh ngôn ngữ.

4.3. Đánh giá hiệu năng Truy xuất và Xếp hạng định lượng

Phần này phân tích kết quả thực nghiệm để xác nhận vai trò của từng ánh xạ trong việc tối đa hóa recall ở tầng truy xuất và tối ưu precision ở tầng xếp hạng.

Do quy mô tập dữ liệu thực nghiệm nằm ở mức trung bình (2.000 hồ sơ), nhằm tránh hiện tượng sai lệch do phân chia dữ liệu ngẫu nhiên (sampling bias) và đảm bảo tính tổng quát của các chỉ số hiệu năng, nghiên cứu áp dụng chiến lược Kiểm chứng chéo 5 lớp (5-fold Cross-Validation). Cụ thể, tập dữ liệu được chia thành 5 phần (folds) bằng nhau. Trong mỗi lượt thực nghiệm, hệ thống luân phiên sử dụng 4 phần (1.600 hồ sơ) để huấn luyện mô hình LambdaMART và phần còn lại (400 hồ sơ) để đánh giá độc lập. Kết quả cuối cùng trình bày tại Bảng 4.1. là giá trị trung bình cộng của các lần lặp. Phương pháp này đảm bảo rằng mọi hồ sơ ứng viên đều được tham gia vào cả quá trình huấn luyện và kiểm thử, giúp các độ đo NDCG và MAP phản ánh chính xác năng lực thực sự của pipeline đề xuất thay vì phụ thuộc vào một tập Test-set cố định.

Bảng 4.1. So sánh hiệu năng của các phương pháp Truy xuất và Tái xếp hạng

Cấu hình	NDCG@5	NDCG@10	MAP@10	MRR
$f_{\text{retrieval}} = \text{BM25} \forall d \in D$	0.834	0.878	0.090	1.000
$f_{\text{retrieval}} = \text{Dense} \forall d \in D$	0.834	0.804	0.082	0.740
$f_{\text{rerank}} = f_{\text{LTR}} \forall d \in D$	0.766	0.805	0.083	0.767
$f_{\text{rerank}} = f_{\text{CE}} \forall d \in D$	0.884	0.867	0.092	1.000
$f_{\text{RRF}} \rightarrow \text{Top-}K \rightarrow$ $f_{\text{fusion}}(f_{\text{CE}}, f_{\text{LTR}})$	0.843	0.869	0.092	0.900

Đánh giá kết quả thực nghiệm đối với nhóm IR Metrics:

- **Rào cản độ trễ của Cross-Encoder toàn cục (Brute-force):** Kết quả cho thấy khi áp dụng Cross-Encoder quét trực tiếp trên toàn bộ không gian dữ liệu (toàn bộ corpus), mô hình đạt đỉnh NDCG@5 (0.884). Điều này chứng minh sức mạnh vượt trội của kiến trúc attention ở cấp độ token (token-level interaction) trong

việc hiểu ngữ nghĩa sâu. Tuy nhiên, cái giá phải trả là cực kỳ đắt: Việc bắt cặp và suy luận mọi tổ hợp (JD, CV) khiến độ phức tạp tính toán tăng tuyến tính $\mathcal{O}(|D|)$ với chi phí phần cứng khổng lồ. Cách tiếp cận "vét cạn" này hoàn toàn phá vỡ ràng buộc về độ trễ (latency) của một hệ thống thời gian thực, khiến nó không thể triển khai độc lập trong thực tế.

- **Hạn chế của LambdaMART trên không gian lớn:** Khi chạy LambdaMART độc lập trên toàn bộ tập dữ liệu, hiệu năng lại sụt giảm ($\text{NDCG@10} = 0.805$). Nguyên nhân không chỉ do quy mô dữ liệu huấn luyện chưa đủ để mô hình hội tụ và vượt qua rào cản từ khóa (exact-match) của miền dữ liệu IT, mà còn do việc ép mô hình học xếp hạng trên một không gian quá lớn chứa đầy các hồ sơ "nhiều" (hoàn toàn không liên quan đến truy vấn) đã làm chệch hướng các lambda gradients, dẫn đến các cây quyết định trong kiến trúc Gradient Boosting bị tối ưu hóa sai lệch, khó phân biệt rạch ròi các đặc trưng tinh vi trong feature vector $\phi(q, d)$.
- **Sự trượt chuẩn của Dense Retrieval trong miền đặc thù:** So sánh giữa hai đường cơ sở cho thấy thuật toán từ vựng cổ điển BM25 đạt kết quả cao hơn mô hình học sâu ngữ nghĩa BGE-large. Điều này củng cố lập luận: Trong miền dữ liệu đặc thù như IT, sự trùng khớp chính xác của các thuật ngữ chuyên ngành (như "PostgreSQL", "React") mang lại lợi thế cho các phương pháp khớp từ vựng. Ngược lại, hiệu năng thấp hơn của Dense Retrieval có thể xuất phát từ vấn đề mô hình chưa được tinh chỉnh (Domain Adaptation) cho dữ liệu tuyển dụng.
- **Sự ưu việt của Kiến trúc lai phân tầng đề xuất:** Luồng kiến trúc đề xuất đã chứng minh được tính đúng đắn của chiến lược coarse-to-fine. Dù NDCG@5 (0.843) thấp hơn một chút so với việc dùng Cross-Encoder vét cạn toàn bộ corpus, nhưng kiến trúc đề xuất lại đạt **MAP@10 cao nhất toàn bảng (0.092)**, thể hiện khả năng bao phủ và duy trì độ ổn định tuyệt vời. Quan trọng nhất, bằng cách dùng Hybrid-RRF làm "phễu lọc", hệ thống đã thu hẹp không gian dữ liệu trước khi đẩy vào các mô hình nặng, qua đó đạt được chất lượng xếp hạng tiệm cận với Cross-Encoder toàn cục nhưng với **chi phí tính toán và độ trễ chỉ bằng một phần rất nhỏ**. Đây là minh chứng rõ nét cho một thiết kế hệ thống tối ưu, cân bằng hoàn hảo giữa độ chính xác và khả năng triển khai thực tiễn.

4.4. Đánh giá cơ chế sinh ngôn ngữ có ràng buộc (Faithful RAG)

Thực nghiệm này được thiết kế nhằm đánh giá hiệu quả của ánh xạ g_{rag} đã định nghĩa trước đó, thông qua việc so sánh giữa cấu hình ràng buộc và cấu hình Faithful RAG đề xuất. Cụ thể, nghiên cứu tiến hành thực nghiệm trên tập mẫu gồm 100 cặp hồ sơ và truy vấn tên hai cấu hình của cùng một mô hình nền tảng (GPT-4o-mini):

- Cấu hình Vanilla (baseline): Không áp dụng các ràng buộc prompt và không cấu hình τ .
- Cấu hình đề xuất: Áp dụng chiến lược bối cảnh có cấu trúc JSON, thiết lập $\tau = 0.2$ và sử dụng hệ thống prompt ràng buộc phủ định đã mô tả ở Chương 3.

Bên cạnh đó, nhằm loại bỏ thiên lệch do việc chọn mẫu (sampling bias) và đảm bảo tính tổng quát (generalizability) của các chỉ số RAGAS trên toàn bộ không gian dữ liệu, thực nghiệm được thiết kế theo chiến lược bốc mẫu ngẫu nhiên lặp lại (Repeated Random Sub-sampling). Cụ thể, quá trình trích xuất ngẫu nhiên 100 mẫu kiểm thử và đánh giá bằng framework RAGAS được tiến hành lặp lại độc lập 5 lần (tạo ra 5 tập con dữ liệu khác nhau). Kết quả báo cáo tại Bảng 4.2 là giá trị trung bình cộng của 5 phiên đánh giá này. Cách tiếp cận khắt khe này là minh chứng thống kê vững chắc cho thấy khả năng kiểm soát ảo giác của hệ thống không phụ thuộc vào sự phân bố ngẫu nhiên của tập mẫu, mà thực sự là đặc tính bền vững (robustness) của kiến trúc Faithful RAG đã đề xuất:

Bảng 4.2. So sánh nhóm độ đo RAGAS

Độ đo RAGAS	Faithfulness	Context Precision	Context Recall	Answer Relevancy
Cấu hình Vanilla	0.827	0.773	0.791	0.735
Cấu hình đề xuất	0.977	0.810	0.810	0.706

- **Hiệu quả kiểm soát ảo giác:** Kết quả thực nghiệm cho thấy mặc dù cùng sử dụng một mô hình nền tảng, nhưng việc thay đổi thiết kế ánh xạ g_{rag} đã tạo ra sự khác biệt đáng kể về độ trung thực. Ở cấu hình Vanilla, mô hình bộc lộ rõ xu hướng "nội suy sáng tạo" (hallucination) khi đối mặt với văn bản thô nhiều nhiễu. Việc áp dụng chiến lược thiết kế của đề tài đã giúp nâng chỉ số Faithfulness đáng kể. Do f_{gen} được giữ cố định, sự cải thiện này phản ánh trực tiếp hiệu quả của cơ

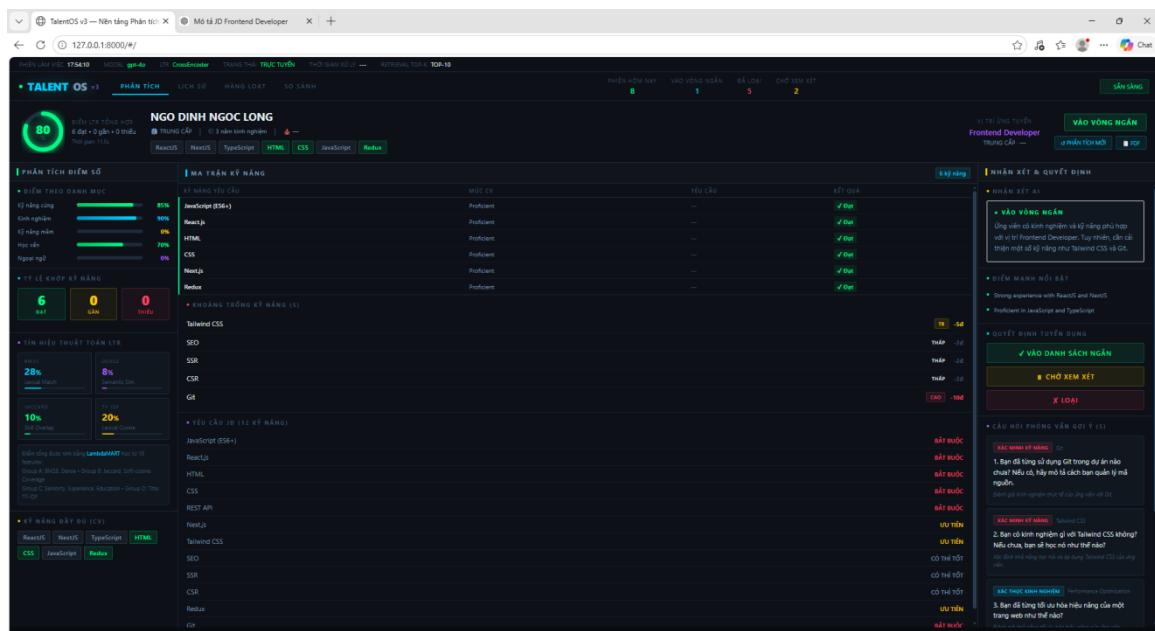
chế ràng buộc ngữ cảnh trong g_{rag} , cho thấy thiết kế hệ thống đóng vai trò quan trọng bên cạnh năng lực của mô hình nền tảng.

- **Tối ưu hóa khả năng tập trung:** Chỉ số Context Precision tăng từ 0.773 lên 0.810 minh chứng rằng việc ép dữ liệu qua "phễu" JSON giúp mô hình loại bỏ các thông tin gây nhiễu, tập trung hoàn toàn vào các đặc trưng nghiệp vụ cốt lõi.
- **Sự đánh đổi khoa học (Scientific Trade-off):** Việc áp dụng ràng buộc ngữ cảnh giúp cải thiện đáng kể Faithfulness, nhưng dẫn đến giảm nhẹ Answer Relevancy. Nguyên nhân là do mô hình bị giới hạn chỉ sử dụng thông tin từ ngữ cảnh truy xuất, nên không thể suy diễn để “làm đầy” câu trả lời khi ngữ cảnh chưa bao phủ đầy đủ truy vấn. Ngược lại, cấu hình không ràng buộc có thể tạo ra các câu trả lời có vẻ liên quan hơn, nhưng tiềm ẩn nguy cơ sinh thông tin không có cơ sở. Do đó, sự đánh đổi này phản ánh ưu tiên của hệ thống đối với tính trung thực và khả năng truy vết thông tin.

4.5. Phân tích định tính qua trường hợp kiểm thử thực tế (qualitative case study)

Để chứng minh các chỉ số đánh giá định lượng thực sự mang lại giá trị trong môi trường vận hành thực tiễn, nghiên cứu tiến hành phân tích định tính đầu ra của hệ thống TalentOS thông qua giao diện người dùng (User Interface, UI).

4.5.1. Năng lực đối sánh chuyên sâu (Single CV Analysis)



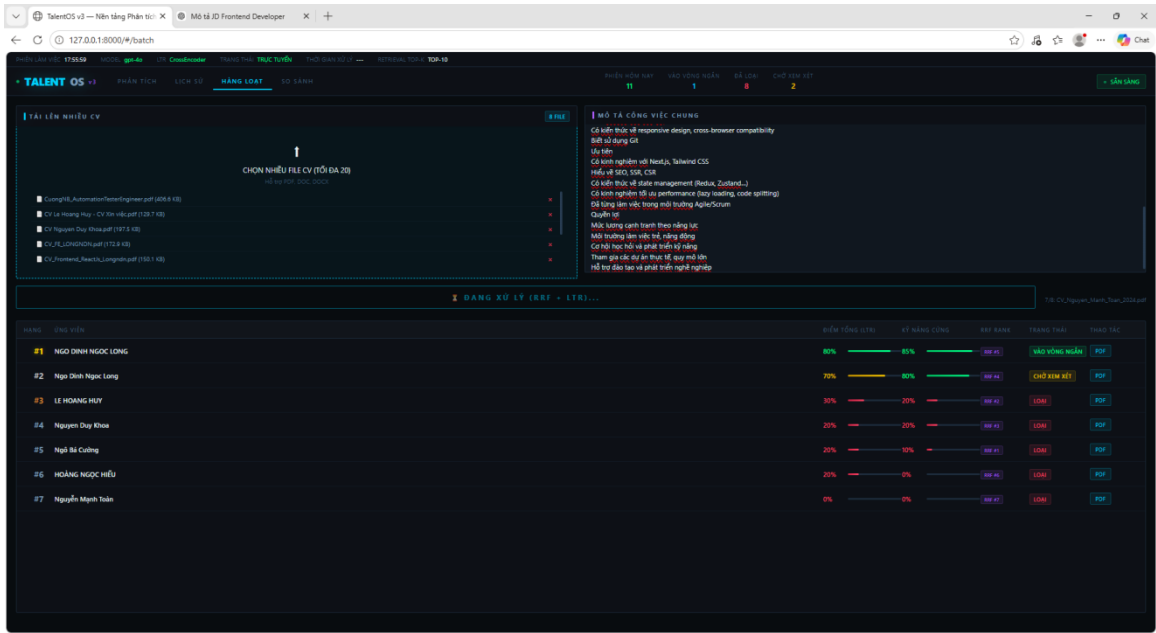
Hình 4.1. Giao diện trực quan hóa kết quả đối sánh đa chiều và suy luận AI của hệ thống đối với một hồ sơ thực tế

Nguồn: Tác giả tự tổng hợp

Như minh chứng tại Hình 4.1, hệ thống đã triển khai thành công cơ chế **suy luận có thể truy vết (Traceable Reasoning)**, chuyển hóa các tín hiệu toán học phức tạp thành thông tin hỗ trợ quyết định (Decision Support) trực quan:

- **Tính minh bạch của Tầng xếp hạng:** Cột bên trái hiển thị rõ ràng các thành phần cấu thành điểm số, bao gồm phân rã theo nhóm đặc trưng (Hard skills, Soft skills, Experience) cùng các tín hiệu định lượng tương ứng. Các chỉ số này được suy ra từ mô hình LambdaMART đã huấn luyện, nhằm cung cấp một góc nhìn diễn giải về vai trò tương đối của từng nhóm đặc trưng trong quá trình xếp hạng. Nhờ đó, hệ thống hỗ trợ nhà tuyển dụng tiếp cận kết quả theo hướng có thể giải thích (interpretable), thay vì chỉ cung cấp một điểm số tổng hợp thiếu ngữ cảnh.
- **Hạn chế ảo giác (Hallucination Control):** Ở cột giữa (Khoảng trống kỹ năng - Skill Gaps), hệ thống nhận diện chính xác các kỹ năng ứng viên còn thiếu so với JD và đánh dấu ưu tiên một cách trung thực. LLM tuyệt đối không bịa đặt kinh nghiệm cho ứng viên, minh chứng cho chỉ số Faithfulness 0.977 đã đạt được.
- **Lập luận có cơ sở (Grounded Verdict):** Cột bên phải thể hiện nhận xét của AI (AI Verdict) bám sát vào kết quả đối chiếu số liệu. Bộ câu hỏi phỏng vấn được sinh ra không mang tính rập khuôn mà nhắm trực tiếp vào những kỹ năng mà ứng viên ứng tuyển, giúp HR khai thác sâu đúng trọng tâm chuyên môn.

4.5.2. Năng lực xử lý hàng loạt theo thời gian thực (Batch Streaming)

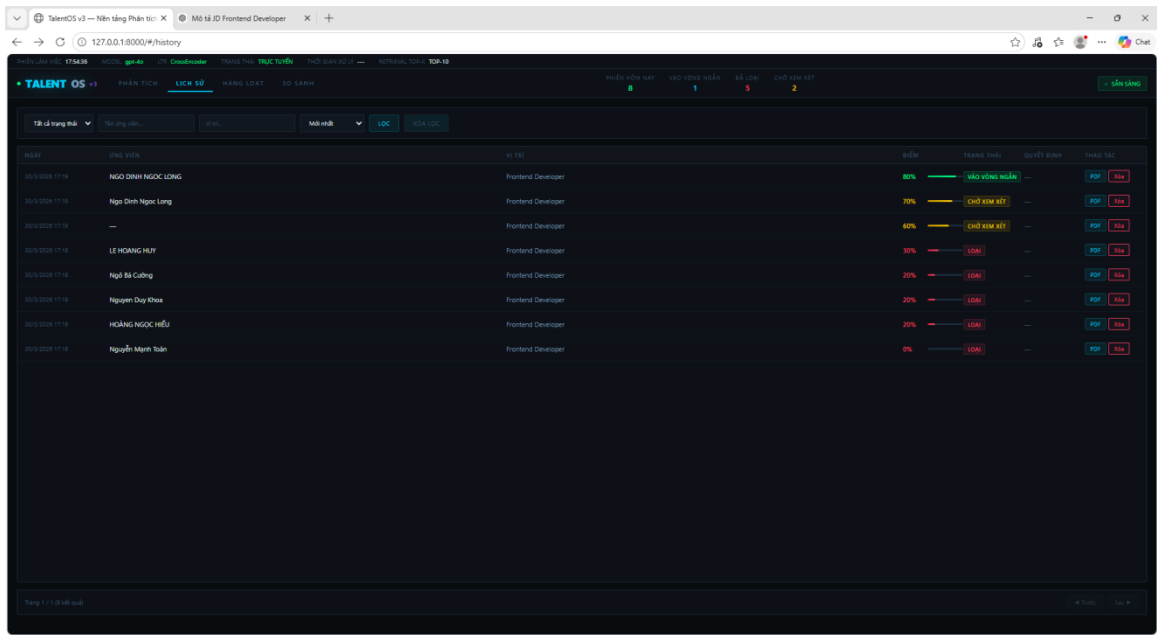


Hình 4.2. Giao diện xử lý phân tích hàng loạt tích hợp luồng dữ liệu thời gian thực (SSE)

Nguồn: Tác giả tự tổng hợp

Hình 4.2 là minh chứng cho thấy kiến trúc hệ thống xử lý bất đồng bộ (Asynchronous Architecture) đã vận hành thành công. Đối với nghiệp vụ phân tích hàng loạt, hệ thống không bị đóng băng mà liên tục trả về kết quả thông qua cơ chế Server-Sent Events (SSE). Bảng xếp hạng cập nhật theo thời gian thực (Real-time update) với các ứng viên được sắp xếp theo thứ tự điểm số giảm dần, cho phép nhà tuyển dụng nắm bắt ngay lập tức bức tranh toàn cảnh của đợt tuyển dụng mà không cần chờ đợi toàn bộ lô dữ liệu hoàn tất.

4.5.3. Năng lực quản trị vòng đời dữ liệu (Session Management)



Hình 4.3. Giao diện quản lý lịch sử các phiên phân tích và lưu trữ quyết định nhân sự
 Nguồn: Tác giả tự tổng hợp

Bên cạnh sức mạnh của lõi AI, Hình 4.3 cho thấy hệ thống đáp ứng đầy đủ các yêu cầu phi chức năng (Non-Functional Requirements) về quản trị dữ liệu. Mọi phiên phân tích đều được lưu trữ bền vững trên cơ sở dữ liệu (PostgreSQL với định dạng JSONB). Hệ thống cung cấp khả năng truy vết, bộ lọc đa chiều (theo Trạng thái, Vị trí) và đặc biệt là cơ chế ghi nhận quyết định cuối cùng của con người (Human-in-the-loop) tách biệt hoàn toàn với khuyến nghị ban đầu của AI (AI Recommendation). Thiết kế này tạo tiền đề quan trọng cho việc thu thập dữ liệu phản hồi (Feedback loop) nhằm tái huấn luyện mô hình Learning-to-Rank trong tương lai.

CHƯƠNG 5:

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Tổng kết và trả lời các câu hỏi nghiên cứu

Dựa trên nền tảng lý thuyết và kết quả thực nghiệm, nghiên cứu đã giải quyết trọn vẹn các câu hỏi nghiên cứu (RQ) được đặt ra ở đầu đề tài:

- **Trả lời RQ1: Phương pháp truy xuất nào mang lại hiệu quả tối ưu trong bài toán sàng lọc hồ sơ?**

Kết quả thực nghiệm cho thấy, trong miền dữ liệu tuyển dụng công nghệ thông tin – nơi các thuật ngữ chuyên ngành mang tính định danh cao – phương pháp truy xuất từ vựng cổ điển (BM25) đạt hiệu năng vượt trội so với các phương pháp truy xuất ngữ nghĩa (Dense Retrieval). Tuy nhiên, xét trên phương diện toàn hệ thống, phương pháp lai được đề xuất mang lại hiệu quả tối ưu nhất. Cụ thể, cơ chế dung hợp này khai thác đồng thời khả năng đối sánh chính xác của BM25 và độ bao phủ ngữ nghĩa của Dense Retrieval, từ đó tạo ra tập ứng viên (candidate pool) có chất lượng cao và ổn định, làm đầu vào hiệu quả cho các tầng xếp hạng tiếp theo.

- **Trả lời RQ2: Phương pháp Learning-to-Rank có cải thiện chất lượng xếp hạng so với việc sử dụng bộ trọng số thủ công không?**

Mặc dù về mặt lý thuyết, các mô hình Learning-to-Rank (LTR) có khả năng tối ưu hóa đa tiêu chí, kết quả thực nghiệm lại chỉ ra rằng LambdaMART không mang lại cải thiện đáng kể so với các phương pháp đường cơ sở. Kết luận này nhấn mạnh một thực tế quan trọng: hiệu quả của LTR phụ thuộc chặt chẽ vào chất lượng và quy mô của dữ liệu huấn luyện, đặc biệt là các nhãn đánh giá (relevance judgments) có độ tinh chỉnh cao từ chuyên gia. Trong bối cảnh dữ liệu hạn chế, các bộ trọng số thủ công dựa trên tri thức miền (heuristic weights) vẫn thể hiện tính ổn định và độ tin cậy cao hơn.

- **Trả lời RQ3: Việc tích hợp Cross-Encoder có đáng để sử dụng khi xét đến sự đánh đổi giữa độ chính xác và độ trễ (latency)?**

Kết quả cho thấy việc sử dụng Cross-Encoder là hợp lý, với điều kiện được triển khai trong một kiến trúc phù hợp. Cross-Encoder đạt hiệu năng cao nhất ở nhóm kết quả đầu nhờ khả năng mô hình hóa tương tác ở cấp độ token. Tuy nhiên, chi phí tính toán cao khiến phương pháp này không khả thi khi áp dụng trên toàn bộ tập dữ liệu. Do đó, Cross-Encoder chỉ phát huy hiệu quả tối đa khi được đặt ở tầng cuối của kiến trúc coarse-

to-fine, nơi nó thực hiện tái xếp hạng trên một tập ứng viên nhỏ đã được lọc trước, qua đó cân bằng hiệu quả giữa độ chính xác và độ trễ.

- **Trả lời RQ4: Làm thế nào để hệ thống RAG đảm bảo faithfulness và hạn chế hiện tượng sinh thông tin không có cơ sở?**

Nghiên cứu chứng minh rằng hiện tượng sinh thông tin không có cơ sở (hallucination) có thể được kiểm soát hiệu quả thông qua kiến trúc Faithful RAG. Giải pháp đề xuất bao gồm: (1) chuẩn hóa ngữ cảnh đầu vào dưới dạng cấu trúc JSON thay vì văn bản tự do; (2) thiết kế prompt với các ràng buộc phủ định rõ ràng; và (3) điều chỉnh hyperparameter τ ở mức thấp. Cách tiếp cận này cho phép chuyển đổi vai trò của mô hình ngôn ngữ lớn từ một hệ sinh văn bản sang một tác tử suy luận dựa trên bằng chứng.

5.2. Đóng góp của đề tài

5.2.1. Đóng góp về mặt khoa học

Nghiên cứu cung cấp bằng chứng thực nghiệm về sự suy giảm hiệu quả của các mô hình embedding ngữ nghĩa tổng quát khi áp dụng vào các miền dữ liệu có tính định danh cao (như công nghệ thông tin và y tế), qua đó khẳng định vai trò quan trọng của các phương pháp truy xuất từ vựng như BM25. Đồng thời, nghiên cứu đề xuất và kiểm chứng thành công kiến trúc Faithful RAG dựa trên dữ liệu có cấu trúc (JSON), mở ra một hướng tiếp cận hiệu quả và chi phí thấp nhằm kiểm soát hiện tượng hallucination trong các hệ thống AI doanh nghiệp.

5.2.2. Đóng góp về mặt thực tiễn

Hệ thống TalentOS đã được xây dựng thành công với kiến trúc hướng sự kiện (event-driven) kết hợp xử lý bất đồng bộ (async/await), hỗ trợ luồng dữ liệu thời gian thực (SSE). Hệ thống có khả năng triển khai thực tế trong môi trường doanh nghiệp quy mô vừa và nhỏ nhằm tự động hóa quy trình sàng lọc hồ sơ ứng viên.

5.3. Hạn chế và hướng phát triển tương lai

Bên cạnh các kết quả đạt được, nghiên cứu vẫn còn một số hạn chế nhất định, mở ra các hướng phát triển (Future Works) đầy tiềm năng:

- **Tinh chỉnh mô hình Embedding (Domain Adaptation):** Kết quả cho thấy BGE-large chưa thể hiện tốt trong miền dữ liệu IT. Hướng cải thiện là áp dụng

học đối chiếu (contrastive learning) trên tập dữ liệu CV-JD chuyên biệt, đồng thời mở rộng sang các mô hình đa ngôn ngữ để hỗ trợ xử lý tiếng Việt.

- **Xây dựng tập dữ liệu gán nhãn chuyên gia (Human Annotation):** Việc sử dụng Ground Truth tự động khiến LambdaMART chưa có đủ dữ liệu tinh tế để học hỏi. Việc xây dựng một tập dữ liệu gồm khoảng 500 mẫu được gán nhãn thủ công (0-2) bởi các chuyên gia nhân sự sẽ là chìa khóa để giải phóng toàn bộ sức mạnh của mô hình Learning-to-Rank.
- **Nâng cấp đối sánh kỹ năng cứng bằng Đồ thị tri thức (Knowledge Graph):** Hệ thống hiện tại sử dụng chỉ số Jaccard để đo lường độ trùng khớp kỹ năng cứng. Trong tương lai, việc tích hợp Knowledge Graph sẽ giúp hệ thống hiểu được tính phân cấp của kỹ năng IT (ví dụ: ứng viên có "ReactJS" sẽ tự động được đánh giá có nền tảng "JavaScript"), từ đó khắc phục hạn chế của việc so khớp tập hợp thuần túy.
- **Áp dụng kiến trúc ColBERT (Late Interaction):** Mặc dù Cross-Encoder rất chính xác nhưng độ trễ vẫn là một thách thức ở quy mô lớn. Hướng đi tiếp theo là thử nghiệm các kiến trúc mới (ví dụ: ColBERT – một giải pháp trung gian cho phép tính toán trước các token embeddings (giống Dense) nhưng vẫn duy trì được sự tương tác chi tiết ở giai đoạn cuối (giống Cross-Encoder), hứa hẹn giảm độ trễ đáng kể mà vẫn giữ nguyên chất lượng xếp hạng).
- **Học tăng cường từ phản hồi người dùng (Feedback Loop):** Hệ thống hiện tại lưu trữ quyết định cuối cùng của HR vào cơ sở dữ liệu nhưng chưa tận dụng chúng. Việc xây dựng một vòng lặp phản hồi để tự động tinh chỉnh lại trọng số thuật toán dựa trên các quyết định thực tế này sẽ giúp TalentOS trở thành một hệ thống có khả năng học thích ứng (online learning) theo thời gian.
- **Tích hợp hệ thống vào nền tảng quản trị doanh nghiệp:** Để hiện thực hóa tầm nhìn tối ưu hóa toàn diện hoạt động quản lý và vận hành doanh nghiệp, hướng phát triển chiến lược tiếp theo là tích hợp TalentOS như một AI module vào các nền tảng Hoạch định Nguồn lực Doanh nghiệp hoặc Quản trị Nguồn nhân lực tổng thể. Sự tích hợp này sẽ tạo ra một luồng dữ liệu thông suốt: kết quả đối sánh từ khâu tuyển dụng đầu vào sẽ trở thành cơ sở để hệ thống dự báo nhu cầu nhân sự, thiết lập tiêu chí đánh giá hiệu suất (KPIs) và hoạch định lộ trình đào tạo, từ đó mang lại giá trị cộng hưởng lớn cho toàn bộ hệ sinh thái vận hành của tổ chức.

DANH MỤC TÀI LIỆU THAM KHẢO

- Anthropic. (2024). *The Claude 3 model family: Opus, Sonnet, Haiku*.
<https://www.anthropic.com/news/claude-3-family>
- Bajaj, P., Campos, D., Craswell, N., Deng, L., Gao, J., Liu, X., Majumder, R., McNamara, A., Mitra, B., Nguyen, T., Rosenberg, M., Song, X., Stoica, A., Tiwary, S., & Wang, T. (2018). *MS MARCO: A human generated machine reading comprehension dataset*. arXiv.
<https://doi.org/10.48550/arXiv.1611.09268>
- Bassani, E. (2022). ranx: A blazing-fast Python library for ranking evaluation and comparison. Trong *Advances in information retrieval* (tr. 259–264). Springer International Publishing. https://doi.org/10.1007/978-3-030-99739-7_30
- Burges, C. J. C. (2010). *From ranknet to lambdarank to lambdamart: An overview*. Microsoft Research. <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/MSR-TR-2010-82.pdf>
- Cormack, G. V., Clarke, C. L., & Buettcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. Trong *Proceedings of the 32nd international ACM SIGIR conference* (tr. 758–759).
<https://doi.org/10.1145/1571941.1572114>
- Es, S., James, J., Espinosa Anke, L., & Schockaert, S. (2024). RAGAs: Automated evaluation of retrieval augmented generation. Trong *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations* (tr. 150–158).
<https://doi.org/10.18653/v1/2024.eacl-demo.16>
- Heakl, M. A., Gad-ElRab, M. H., & El-Beltagy, S. R. (2024). *ResumeAtlas: Revisiting resume classification with a large-scale dataset and deep learning*. arXiv.
<https://doi.org/10.48550/arXiv.2406.18125>

- Hinton, G., Vinyals, O., & Dean, J. (2015). *Distilling the knowledge in a neural network*. arXiv. <https://doi.org/10.48550/arXiv.1503.02531>
- Järvelin, K., & Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems (TOIS)*, 20(4), 422–446. <https://doi.org/10.1145/582415.582418>
- Johnson, J., Douze, M., & Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>
- Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. T. (2020). Dense passage retrieval for open-domain question answering. Trong *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (tr. 6769–6781). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2020.emnlp-main.550>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W. T., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9475. <https://doi.org/10.48550/arXiv.2005.11401>
- Lin, C. Y. (2004). ROUGE: A package for automatic evaluation of summaries. Trong *Text Summarization Branches Out* (tr. 74–81). Association for Computational Linguistics. <https://aclanthology.org/W04-1013/>
- Liu, T.-Y. (2009). Learning to rank for information retrieval. *Foundations and Trends in Information Retrieval*, 3(3), 225–331. <https://doi.org/10.1561/15000000016>
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239), Bài 2.

- <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>
- Nogueira, R., & Cho, K. (2019). *Passage re-ranking with BERT*. arXiv. <https://doi.org/10.48550/arXiv.1901.04085>
- OpenAI. (2023). *GPT-4 technical report*. arXiv. <https://doi.org/10.48550/arXiv.2303.08774>
- Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). Bleu: A method for automatic evaluation of machine translation. Trong *Proceedings of the 40th annual meeting of the Association for Computational Linguistics* (tr. 311–318). <https://doi.org/10.3115/1073083.1073135>
- Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). *SQuAD: 100,000+ questions for machine comprehension of text*. arXiv. <https://doi.org/10.48550/arXiv.1606.05250>
- Ramírez, S. (2018). *FastAPI framework, high performance, easy to learn, fast to code, ready for production*. <https://fastapi.tiangolo.com/>
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. Trong *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (tr. 3982–3992). Association for Computational Linguistics. <https://doi.org/10.18653/v1/D19-1410>
- Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389. <https://doi.org/10.1561/15000000019>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural*

- Information Processing Systems (NeurIPS)*, 30, 5998–6008. <https://doi.org/10.48550/arXiv.1706.03762>
- Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 5776–5788. <https://doi.org/10.48550/arXiv.2002.10957>
- Wooldridge, M. (2009). *An introduction to multiagent systems* (Lần xuất bản thứ 2). John Wiley & Sons.
- Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., & Zheng, R. (2023). *The rise and potential of large language model based agents: A survey*. arXiv. <https://doi.org/10.48550/arXiv.2309.07864>
- Xiao, S., Liu, Z., Zhang, G., & Muennighoff, N. (2023). *C-Pack: Packaged resources to advance general Chinese embedding*. arXiv. <https://doi.org/10.48550/arXiv.2309.07597>
- Yang, C. (2024, ngày 06 tháng 05). *Building RAG systems with open-source and custom AI models*. BentoML. <https://www.bentoml.com/blog/building-rag-with-open-source-and-custom-ai-models>
- Zhou, Y., Yao, J., Dou, Z., Wu, L. Y., Zhang, P., & Wen, J. R. (2022). *Ultron: An ultimate retriever on corpus with a model-based indexer*. arXiv. <https://doi.org/10.48550/arXiv.2208.09257>

PHỤ LỤC

Phụ lục A: Đặc tả mô hình và lược đồ dữ liệu

Phụ lục B: Đặc tả Cấu hình môi trường và Thư viện cài đặt

Phụ lục C: Đặc tả Giao diện lập trình ứng dụng (REST API)

Phụ lục D: Ma trận truy vết yêu cầu và kiến trúc hệ thống

PHỤ LỤC A

Đặc tả mô hình và lược đồ dữ liệu

A.1. Cấu trúc dữ liệu In-Memory

Các cấu trúc dữ liệu nội bộ được định nghĩa bằng Pydantic và Dataclass để truyền tải ngữ cảnh giữa các module:

```
# 1. Biểu diễn đầu vào
class Document(BaseModel):
    doc_id: str          # Định danh CV (ví dụ: cv_0001)
    chunk_id: str        # doc_id + chỉ số chunk (ví dụ: cv_0001_c0)
    text: str            # Nội dung văn bản của phân đoạn
    metadata: dict       # Lưu thông tin loại phân đoạn

class Query(BaseModel):
    query_id: str        # UUID, sinh tự động mỗi request
    text: str            # Nội dung JD đầy đủ

# 2. Biểu diễn kết quả trung gian và đầu ra
@dataclass
class RetrievalResult:
    doc_id: str
    chunk_id: str
    score: float         # BM25 score hoặc Cross-encoder score
    text: str
    rank: int = 0        # Thứ hạng sau khi sắp xếp
    metadata: dict       # Kế thừa từ Document.metadata

class CandidateResult(BaseModel):
    rank: int
    doc_id: str
    rerank_score: float
    verdict: str         # Nhận xét 2-3 câu về độ phù hợp
    matched_skills: list[str] = []
    skill_gaps: list[str] = []
    recommendation: str = 'hold' # shortlist | hold | reject
    interview_questions: list[dict] = []
    metadata: dict = {}

# 3. API models
class AnalysisRequest(BaseModel):
    jd_text: str         # JD đầu vào

class AnalysisResponse(BaseModel):
    query_id: str
    total_retrieved: int # Số lượng CV được truy xuất bởi BM25
    top_candidates: list[CandidateResult]
```

A.2. Thiết kế bảng cơ sở dữ liệu

Bảng A.1. Cấu trúc bảng sessions lưu trữ lịch sử phân tích

Cột dữ liệu	Kiểu dữ liệu	Mô tả chi tiết
session_id	VARCHAR(64) PK	UUID định danh duy nhất mỗi phiên phân tích
filename	TEXT	Tên tệp CV gốc được tải lên (nullable)

candidate_name	TEXT (INDEX)	Tên ứng viên, trích xuất từ kết quả LLM
position_title	TEXT (INDEX)	Tên vị trí tuyển dụng, trích từ JD
overall_score	FLOAT	Điểm phù hợp tổng thể $\in [0, 100]$
recommendation	VARCHAR(20) (INDEX)	Khuyến nghị tuyển dụng: shortlist hold reject
decision	VARCHAR(20)	Quyết định cuối của HR (nullable, cập nhật sau)
created_at	VARCHAR(32) (INDEX)	Thời gian tạo, định dạng ISO-8601
processing_time_ms	INTEGER	Thời gian xử lý toàn bộ pipeline (ms)
full_result	JSONB	Toàn bộ kết quả phân tích dạng JSON

Các cột candidate_name, position_title, recommendation và created_at được đánh chỉ mục để tối ưu hóa các truy vấn lọc và sắp xếp phổ biến trong chức năng quản lý lịch sử. Cột decision được tách riêng khỏi recommendation để phân biệt giữa gợi ý của AI và quyết định thực tế của HR.

A.3. Lược đồ JSON lưu trữ kết quả đầu ra của LLM

```
# cv_data: trích xuất thông tin
class Document(BaseModel):
    doc_id: str          # Định danh CV (ví dụ: cv_0001)
    chunk_id: str        # doc_id + chỉ số chunk (ví dụ: cv_0001_c0)
    text: str            # Nội dung văn bản của phân đoạn
    metadata: dict       # Lưu thông tin loại phân đoạn
class Query(BaseModel):
    query_id: str        # UUID, sinh tự động mỗi request
    text: str            # Nội dung JD đầy đủ

# match_result: kết quả đối chiếu và lập luận
{
  "overall_score": 0.0,
  "category_scores": {
    "hard_skills": 0.0,
    "soft_skills": 0.0,
    "experience": 0.0,
    "education": 0.0,
    "language": 0.0
  },
  "matched_skills": [{"skill": "string", "status": "match|partial", "cv_level":
"string"}],
  "skill_gaps": [{"skill": "string", "priority": "high|medium|low",
"impact_score": 0.0}],
  "key_strengths": ["string"],
  "ai_verdict": "string",
  "recommendation": "shortlist|hold|reject"
}
```

PHỤ LỤC B**Cấu hình môi trường và thư viện cài đặt****B.1. Môi trường phát triển và triển khai***Bảng B.1. Môi trường phát triển và triển khai*

Thành phần	Phiên bản	Hệ điều hành	Ghi chú
Python	3.11	Linux (slim)	Runtime chính của toàn bộ backend
Docker	≥ 24.0	Linux/macOS/Windows	Đóng gói và triển khai hệ thống
Docker Compose	≥ 2.0	Linux	Quản lý service đa container
Poppler-utils	Hệ thống	Linux (apt)	Thư viện C++ hỗ trợ pdfplumber đọc PDF

B.2. Thư viện và framework dùng cho Backend*Bảng B.2. Thư viện backend theo giai đoạn pipeline*

Thư viện	Phiên bản	Vai trò
fastapi	≥ 0.110	Web framework async, tự sinh OpenAPI docs
uvicorn	≥ 0.29	ASGI server chạy FastAPI, hỗ trợ reload
python-multipart	$\geq 0.0.9$	Parse multipart/form-data cho file upload
pydantic	≥ 2.0	Định nghĩa và validate schema dữ liệu
pydantic-settings	≥ 2.0	Đọc cấu hình từ biến môi trường /.env
pdfplumber	≥ 0.10	Trích xuất văn bản từ file PDF

python-docx	≥1.1	Trích xuất văn bản từ file DOC/DOCX
rank-bm25	≥0.2.2	Thuật toán BM25 Okapi cho lexical retrieval
sentence-transformers	≥2.6	CrossEncoder reranking dựa trên BERT
numpy	≥1.24	Xử lý mảng điểm số từ CrossEncoder
langchain	≥0.1	Orchestration gọi LLM, quản lý message
langchain-openai	≥0.1	Tích hợp OpenAI GPT qua LangChain
langchain-anthropic	≥0.1	Tích hợp Anthropic Claude qua LangChain

B.3. Thư viện và framework dùng cho Frontend

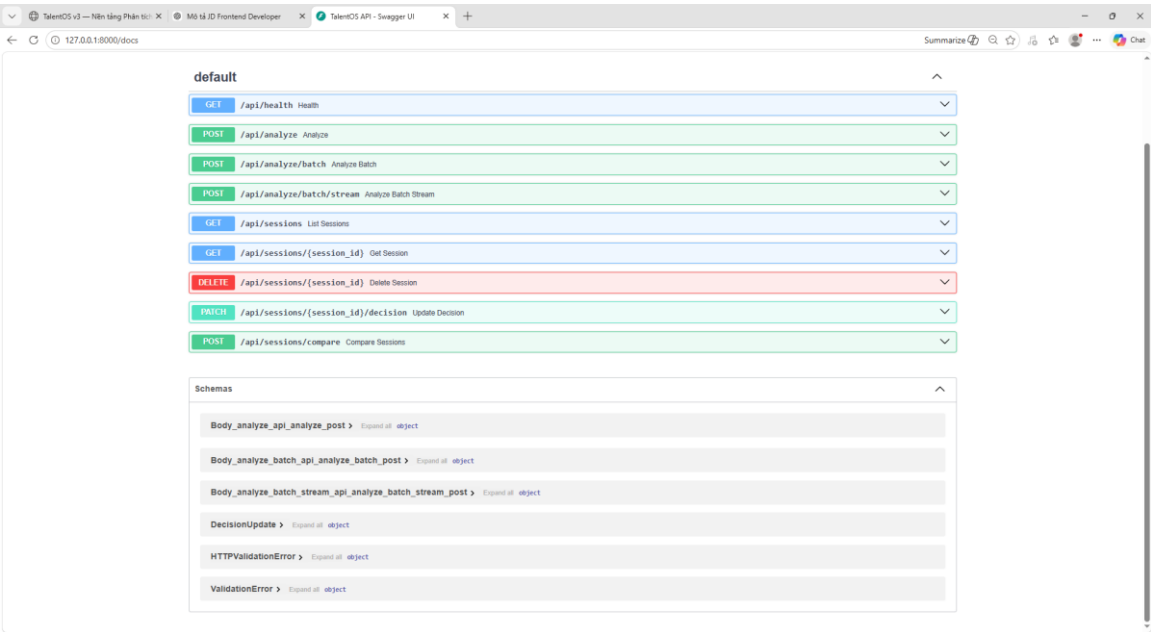
Bảng B.3. Công nghệ dùng cho Frontend

Công nghệ	Loại	Vai trò
HTML5 / CSS3	Markup & Style	Cấu trúc và định dạng giao diện
Vanilla JavaScript (ES6+)	Script	Xử lý logic UI, routing, gọi API
Fetch API / EventSource	Web API	Gọi REST API và nhận SSE streaming từ backend
Static file serving	FastAPI mount	Backend tự phục vụ frontend qua StaticFiles, không cần web server riêng

PHỤ LỤC C

Đặc tả giao diện lập trình ứng dụng (rest api)

Nhờ ứng dụng framework FastAPI, toàn bộ các Endpoints của hệ thống TalentOS được tự động lập tài liệu theo chuẩn OpenAPI.



Hình C.1. Giao diện tài liệu API tương tác tự động sinh bởi Swagger UI

Nguồn: Tác giả tự tổng hợp

Bảng C.1. Tổng hợp các API Endpoint

Method	URL	Chức năng
GET	/api/health	Kiểm tra trạng thái, phiên bản, cấu hình
POST	/api/analyze	Phân tích CV đơn lẻ (multipart/form-data)
POST	/api/analyze/batch	Phân tích batch đồng bộ
POST	/api/analyze/batch/stream	Phân tích batch streaming SSE
GET	/api/sessions	Danh sách lịch sử, filter + sort + pagination
GET	/api/sessions/{session_id}	Chi tiết một phiên (full_result JSONB)
PATCH	/api/sessions/{session_id}/decision	Cập nhật quyết định HR
DELETE	/api/sessions/{session_id}	Xóa phiên

PHỤ LỤC D

Ma trận truy vết yêu cầu và kiến trúc hệ thống

Bảng D.1. Ma trận truy vết yêu cầu và kiến trúc hệ thống

Mã Yêu Cầu	Vấn Đề Nghiệp Vụ	Giải Pháp Thiết Kế Kiến Trúc & Thuật Toán Đáp Ứng
[FR-01]	Phân tích CV đơn lẻ	Xây dựng Pipeline đa tầng hoàn chỉnh.
[FR-02]	Phân tích hàng loạt (Batch processing)	Áp dụng chiến lược tối ưu Coarse-to-fine.
[FR-03]	Phân tích theo luồng (Streaming)	Triển khai kiến trúc hướng sự kiện, sử dụng hàng đợi bất đồng bộ và giao thức truyền tải thời gian thực.
[FR-04] [FR-05]	Quản lý lịch sử & Cập nhật quyết định	Sử dụng chiến lược phi chuẩn hóa với định dạng JSONB trên hệ quản trị cơ sở dữ liệu PostgreSQL để linh hoạt lưu trữ kết quả phân tích bán cấu trúc.
[NFR-01]	Hiệu năng xử lý	Tuần tự hóa chỉ mục và nạp toàn bộ lên RAM để xóa bỏ nút thắt I/O & Áp dụng cơ chế Model Pre-warming để triệt tiêu độ trễ khởi tạo (cold start).
[NFR-02]	Khả năng mở rộng (Scalability)	Cấu trúc hệ thống theo mô hình Đa tác tử (Multi-Agent System), phân tách rõ ràng tầng định lượng (Retrieval/Ranking) và định tính (Generation).
[NFR-03]	Độ tin cậy (Reliability)	Tích hợp cơ chế Retry & Backoff khi giao tiếp với các dịch vụ LLM bên ngoài.
[NFR-04]	Kiểm soát ảo giác (Faithfulness)	Áp dụng kiến trúc Faithful RAG.
[NFR-05]	Khả năng triển khai (Deployability)	Đóng gói toàn bộ môi trường thực thi bằng nền tảng container hóa (Docker/Docker Compose).
[NFR-06]	Bảo mật & Riêng tư (Data Privacy)	Giao thức dữ liệu nội bộ chỉ lưu trữ và truyền tải dữ liệu văn bản đã được trích xuất.